

Elettronica Digitale

Appunti del corso

Riferimento: Katz, Borriello – *Contemporary Logic Design*

Indice

Capitolo 1. Segnali, algebra di Boole e reti logiche

(27 febbraio – 2 marzo 2026)

1.1 Segnali analogici e digitali

Segnale analogico: assume valori continui (onde). È il tipo di segnale presente in natura.
Segnale digitale: assume valori discreti, tipicamente $\{0, 1\}$.

Il passaggio da un segnale analogico a uno digitale permette di **memorizzare meglio i dati**: un valore discreto è più robusto al rumore e più facile da elaborare e conservare rispetto a un valore continuo.

1.2 Algebra di Boole

L'algebra di Boole fu pubblicata a metà del 1800. Per definirla servono i seguenti ingredienti:

- un insieme B che contiene **2 valori** (es. $\{0, 1\}$);
- su B agiscono **2 operazioni** (tipicamente $+$ e $\cdot$, cioè OR e AND);
- gli **elementi neutri** delle due operazioni sono gli stessi 2 elementi di B ;
- si definisce un'ulteriore operazione, il **NOT**, che manda i due valori l'uno nell'altro ($0 \rightarrow 1$, $1 \rightarrow 0$).

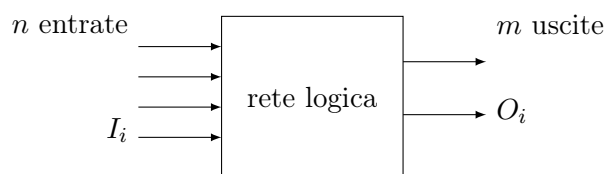
Notazione: la negazione di una variabile si indica con un trattino sopra la lettera, ad esempio $\overline{A}$.

1.3 Cenni storici

- **Shannon** ha inventato le porte logiche.
- **Huntington** ha dimostrato che ogni rete logica può essere implementata usando solo porte a 2 ingressi e 1 uscita.
- Il **transistor** (*transfer + resistor*) nasce e, nel giro di circa 20 anni, viene costruito il primo circuito integrato.

1.4 Reti logiche

Una rete logica può essere vista come una scatola nera con n ingressi I_i e m uscite O_i :



$O_i = f(I_i)$: per ogni uscita esiste una funzione che dipende dagli ingressi I_i ; tale funzione è **suriettiva**. Sia gli O_i che gli I_i possono assumere solo i valori 0 oppure 1.

1.4.1 Funzioni a 1 ingresso

Con un solo ingresso I esistono $2^{2^1} = 4$ possibili funzioni di uscita:

I	O_1	O_2	O_3	O_4
0	0	1	0	1
1	0	0	1	1
	0	NOT	ID	1

Questa è la **tabella di verità** delle 4 funzioni a un ingresso: due sono costanti (0 e 1), una è l'**identità** (ID, o *buffer*) e una è la **negazione** (NOT).

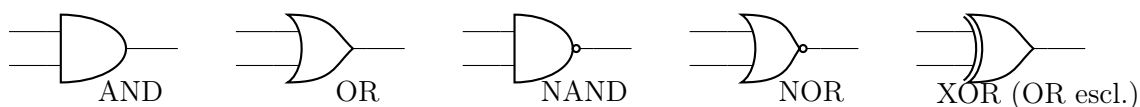


1.4.2 Funzioni a 2 ingressi

Con due ingressi I_1, I_0 ci sono $2^2 = 4$ combinazioni in ingresso e $2^{2^2} = 16$ possibili funzioni di uscita. La tabella completa è:

I_1	I_0	O_1	O_2	O_3	O_4	O_5	O_6	O_7	O_8	O_9	O_{10}	O_{11}	O_{12}	O_{13}	O_{14}	O_{15}	O_{16}
0	0	0	1	0	0	0	1	1	1	0	0	0	1	1	0	1	1
0	1	0	0	1	0	0	1	0	0	1	1	0	1	1	1	0	1
1	0	0	0	0	1	0	0	1	0	1	0	1	1	0	1	1	1
1	1	0	0	0	0	1	0	0	1	0	1	1	0	1	1	1	1
		0	NOR	AND				XOR				NAND				OR	1

Tra le 16 funzioni, le più importanti hanno un nome ed un simbolo proprio:



1.4.3 Funzioni a n ingressi

In generale, con n ingressi si hanno 2^n possibili combinazioni in ingresso e 2^{2^n} possibili funzioni di uscita. Le porte AND, NAND, OR, NOR, XOR si generalizzano naturalmente a n ingressi.

1.5 Bit di parità

Il **bit di parità** è un parametro usato per controllare che un segnale non sia stato alterato durante una trasmissione:

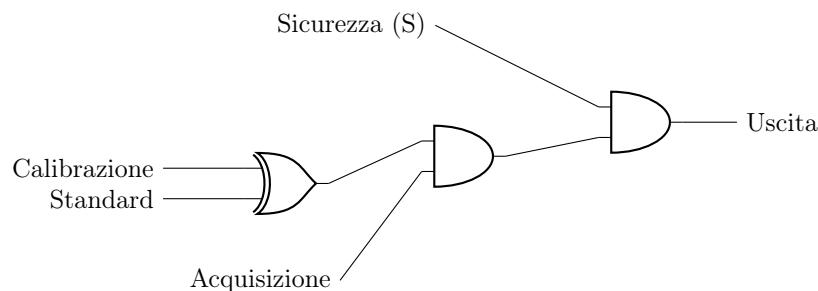
- se il numero di 1 nel messaggio è **pari** $\Rightarrow P = 0$;
- se il numero di 1 nel messaggio è **dispari** $\Rightarrow P = 1$.

1.6 Esercizio: semplice circuito per abilitare la presa dati

Disegnare un circuito che generi un segnale per abilitare la presa dati di un esperimento se ha il segnale di **sicurezza** S attivo, se c'è la richiesta di una presa dati che può essere di **calibrazione** o **standard** (non entrambe), e se l'operatore chiede di abilitare l'**acquisizione**. Inoltre la presa dati si deve arrestare se S non è più attivo.

La condizione “calibrazione o standard, ma non entrambe” è esattamente una **XOR**; il fatto che la presa dati debba restare attiva solo mentre S è alto (e fermarsi non appena S si disattiva) richiede un **AND** con il segnale di sicurezza (non un semplice impulso, ma il livello stesso di S):

$$\text{Uscita} = S \cdot \text{Acquisizione} \cdot (\text{Calibrazione} \oplus \text{Standard})$$



Nota dell'autore degli appunti: gli ingressi potrebbero anche essere invertiti, ma mettendo il segnale di sicurezza sull'ultima porta AND si ottiene un ritardo minore nello spegnimento della presa dati.

Capitolo 2. Realizzazione fisica delle porte logiche

(6 marzo 2026)

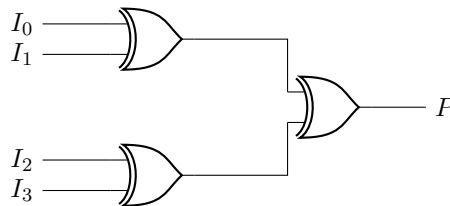
2.1 Esercizio: bit di parità a 4 bit

Progettare un circuito che calcoli il bit di parità di un numero a 4 bit. Il bit di parità è 0 se il numero di 1 nel numero è pari, 1 se è dispari.

Sia $N = I_3I_2I_1I_0$. Si lavora a coppie di bit, applicando lo XOR a multipli di 2:

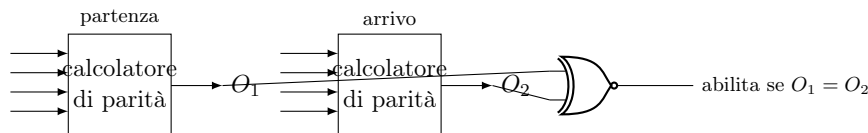
I_3	I_2	XOR	I_1	I_0	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	1	1	0

Il **calcolatore di parità** si ottiene combinando in cascata 3 porte XOR:



2.2 Verifica di una trasmissione tramite parità

Calcolando il bit di parità sia alla **partenza** (O_1) che all'**arrivo** (O_2) e confrontandoli con una porta **XNOR**, si fa passare il segnale solo se $O_1 = O_2$ (cioè 00 oppure 11): questo permette di controllare che il segnale non sia stato alterato durante la trasmissione.



2.3 Caratteristiche fisiche delle porte logiche

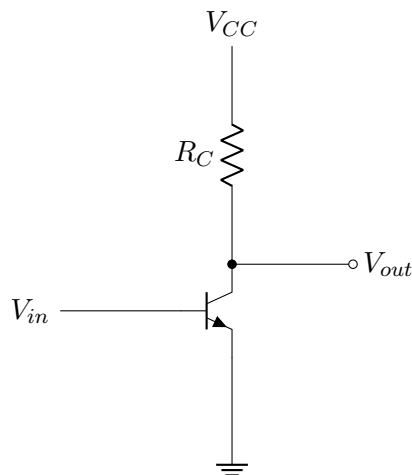
Le diverse **famiglie logiche** (RTL, DTL, HTL, TTL, ECL, MOS, CMOS) si confrontano in base a parametri come la porta di base, il fan-out, la potenza dissipata, l'immunità al rumore, il ritardo di propagazione e la frequenza massima di funzionamento. In generale:

- **TTL**: ottimo compromesso velocità/potenza, molto diffusa;
- **ECL**: velocissima ma con elevato consumo;
- **CMOS**: bassissima potenza dissipata, oggi la più usata nei circuiti integrati.

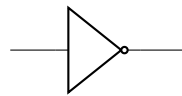
2.4 Implementazione delle porte logiche

2.4.1 NOT con un singolo transistor

La porta NOT più semplice si realizza con un singolo transistor NPN in configurazione a emettitore comune, con una resistenza di pull-up R_C verso V_{CC} :



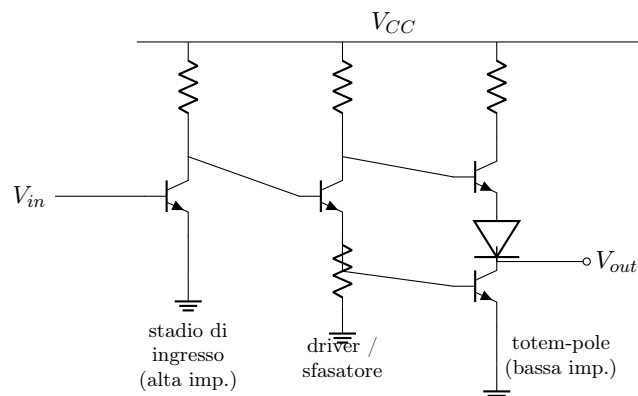
- $V_{in} = 5\text{ V} (= 1) \Rightarrow$ il transistor satura $\Rightarrow V_{out} = 0,2\text{ V} (= 0)$;
- $V_{in} = 0\text{ V} (= 0) \Rightarrow$ il transistor è spento $\Rightarrow V_{out} = 5\text{ V} (= 1)$.



2.4.2 NOT in tecnologia TTL

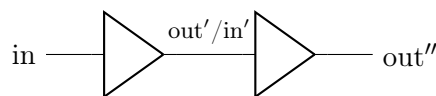
Nella tecnologia TTL (*Transistor-Transistor Logic*) la porta NOT è realizzata con più transistor, organizzati in tre stadi:

1. **stadio di ingresso:** ad alta impedenza;
2. **driver** (o **sfasatore**): pilota i due transistor dello stadio di uscita;
3. **stadio di uscita a totem-pole:** a bassa impedenza, garantisce un'uscita “forte” sia allo stato alto che basso.



2.5 Immunità al rumore

L'immunità al rumore di un sistema digitale non è totale, ma è comunque molto più elevata di quella dei sistemi analogici, perché due porte in cascata “ripuliscono” il segnale ad ogni passaggio:

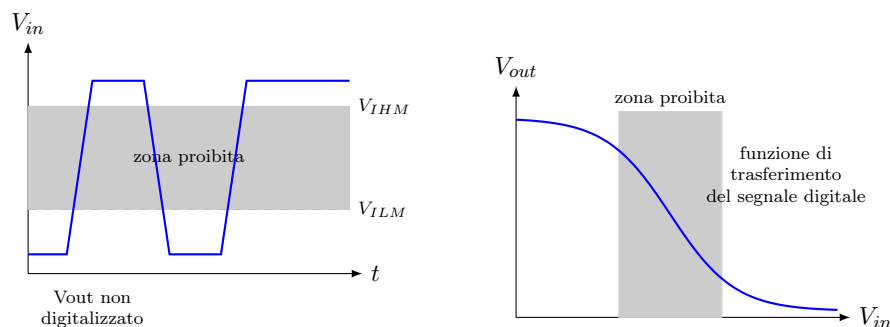


Livello alto	Livello basso
• $V_{OH,min}$: V_{out} minima per uscita alta; • $V_{OH,max}$: V_{out} massima per uscita alta; • $V_{IH,min}$: V_{in} minima per uscita alta; • $V_{IH,max}$: V_{in} massima per uscita alta.	• $V_{OL,min}$: V_{out} minima per uscita bassa; • $V_{OL,max}$: V_{out} massima per uscita bassa; • $V_{IL,min}$: V_{in} minima per uscita bassa; • $V_{IL,max}$: V_{in} massima per uscita bassa.

Si definiscono i margini di rumore (*noise margin*):

$$NM_H = V_{OH,min} - V_{IH,min} \quad NM_L = V_{IL,max} - V_{OL,max}$$

Il *noise margin* rende l'elettronica digitale parzialmente insensibile al rumore: se il segnale resta entro la “zona proibita” tra le soglie, viene comunque interpretato correttamente come 0 o 1 dalla porta successiva, mentre la funzione di trasferimento reale del segnale digitale presenta una zona di transizione dove il comportamento non è garantito.



2.6 Nomenclatura delle famiglie logiche e fan-out

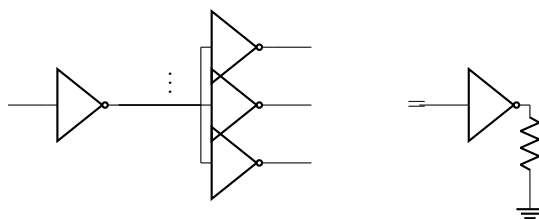
Le sigle delle porte logiche commerciali seguono lo schema:

$$\underbrace{XX}_{\text{ditta produttrice}} \quad \underbrace{NN}_{\text{famiglia logica}} \quad \underbrace{XX}_{\text{funzione}} \quad \underbrace{NN}_{\text{molteplicità}}$$

ad esempio la serie **54** indica componenti per uso militare.

Fan-out = numero massimo di porte che una porta può pilotare in uscita.

Una porta che pilota n porte equivale, dal punto di vista del carico, a una singola porta che pilota un resistore equivalente verso massa:



$$FO_H = \frac{I_{out,H}}{I_{in,H}} \qquad FO_L = \frac{I_{out,L}}{I_{in,L}}$$

Capitolo 3. Algebra di Boole: assiomi, proprietà e sintesi

(13 marzo – 16 marzo 2026)

3.1 Tempi caratteristici delle porte logiche

Tempo di propagazione t_P : calcolato fra V_{in} e V_{out} ; è il tempo che intercorre fra i momenti in cui i segnali raggiungono il 50% della loro intensità fra V_{OL} e V_{OH} . È dovuto alle risposte dei transistor (inter-sat).

Tempo di transizione t_r : calcolato solo fra V_{in} e V_{out} ; è il tempo fra i momenti in cui il segnale è al 10% e al 90% (salita) o al 90% e al 10% (discesa). È dovuto ad effetti capacitivi.

3.2 Assiomi dell'algebra di Boole

L'algebra di Boole è definita dai seguenti assiomi su un insieme B con $|B| = 2$:

- **Chiusura:** $\forall a, b \in B \Rightarrow a + b \in B$ (OR) e $a \cdot b \in B$ (AND).
- **Elementi neutri:** $a + 0 = a$ (0 è neutro per OR); $a \cdot 1 = a$ (1 è neutro per AND).
- **Operazioni complemento:** $\bar{a} + a = 1$ e $\bar{a} \cdot a = 0$ (elemento opposto).
- **Involuzione:** $\bar{\bar{a}} = a$.

3.3 Proprietà dell'algebra di Boole

- **Commutativa:** $a + b = b + a$; $a \cdot b = b \cdot a$.
- **Associativa:** $(a + b) + c = a + b + c$; $(a \cdot b) \cdot c = a \cdot b \cdot c$.
- **Distributiva:** $(a + b) \cdot c = a \cdot c + b \cdot c$; $(a \cdot b) + c = (a + c) \cdot (b + c)$.
Si verifica con le tabelle di verità.
- **Dualità:** posso sempre scambiare un'operazione con l'altra insieme a tutti gli elementi (operazione, neutro, opposto). Funziona perché è un'algebra a 2 elementi, quindi 0 è uno e 1 è l'altro: $a + 0 = a \Rightarrow a \cdot 1 = a$.
- **Assorbimento:** $a + a \cdot b = a$.
- **Idempotenza:** $a + a = a$; $a \cdot a = a$.

3.3.1 Teorema della semplificazione

$$x \cdot y + x \cdot \bar{y} = x$$

$$x \cdot y + x \cdot \bar{y} = x \cdot (y + \bar{y}) = x \cdot 1 = x$$

Duale: $(x + y) \cdot (\bar{x} + \bar{y}) = x \cdot \bar{x} + xy + \bar{x}y + \bar{y}y = 0 + y + y = y$.

3.3.2 Teorema della moltiplicazione e fattorizzazione

$$(x + y) \cdot (\bar{x} + z) = x \cdot z + \bar{x} \cdot y$$

Dimostrazione:

$$(x + y) \cdot (\bar{x} + z) = \bar{x} \cdot x + x \cdot z + \bar{x} \cdot y + y \cdot z = x \cdot z + \bar{x} \cdot y + yz(x + \bar{x}) = x \cdot z(1 + y) + \bar{x} \cdot y(1 + z) = x \cdot z + \bar{x} \cdot y$$

(usando $OR\ Y = 1 \ \forall y = 0, 1$)

Duale: $(\bar{x} \cdot z) + (x \cdot y) = (\bar{x} + y) \cdot (x + z)$.

3.3.3 Leggi di De Morgan

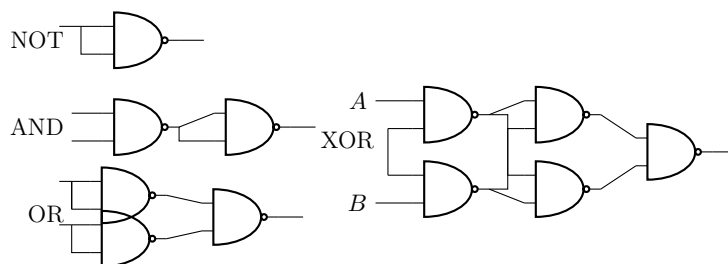
$$x + y + z + \dots = \overline{\bar{x} \cdot \bar{y} \cdot \bar{z} \dots}$$

$$x \cdot y \cdot z \dots = \overline{\bar{x} + \bar{y} + \bar{z} + \dots}$$

3.4 Porte universali: NAND e NOR

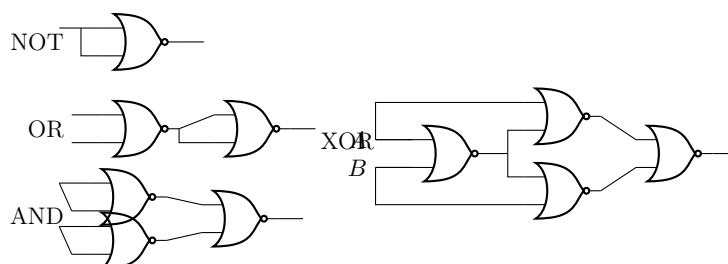
Le porte NAND e NOR sono dette **porte universali** perché con ciascuna di esse è possibile realizzare qualsiasi funzione booleana. AND e OR *non* possono essere porte universali perché non permettono di realizzare le negazioni.

3.4.1 NAND come porta universale



3.4.2 NOR come porta universale

In modo duale alla NAND, anche la NOR può realizzare NOT, AND, OR e XOR:



3.5 Dalla tabella di verità al circuito

Possibili usate di F : data una tabella di verità di $F(a, b, c)$, ogni riga in cui $F = 1$ corrisponde a un **mintermine**; ogni riga in cui $F = 0$ corrisponde a un **maxtermine**.

Mintermine: per costruzione vale sempre 1; è l'AND delle variabili (eventualmente negate) corrispondenti a quella riga.

Maxtermine: per costruzione vale sempre 0; è l'OR delle variabili (eventualmente negate) corrispondenti a quella riga.

Esempio: data la tabella di verità

a	b	c	$F(a, b, c)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Mettendo in OR tutti i mintermini (le righe con $F = 1$) si ottiene la **1ª forma canonica** o **SOP** (*sum of product*):

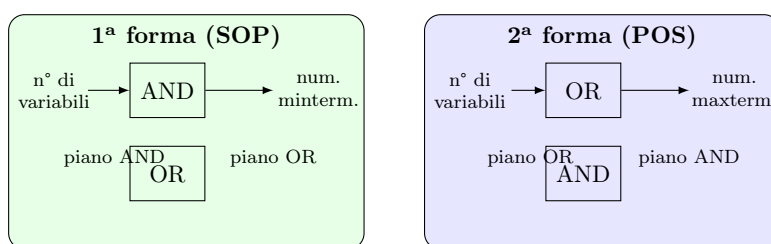
$$F(a, b, c) = \bar{a}\bar{b}c + \bar{a}bc + a\bar{b}c + abc$$

Mettendo in AND tutti i maxtermini (le righe con $F = 0$) si ottiene la **2ª forma canonica** o **POS** (*product of sum*):

$$F(a, b, c) = (a + b + \bar{c}) \cdot (a + \bar{b} + \bar{c}) \cdot (\bar{a} + b + \bar{c}) \cdot (\bar{a} + \bar{b} + c)$$

Una volta ottenuta una delle due forme canoniche, è possibile semplificare l'espressione (ad es. con le mappe di Karnaugh) e costruire il circuito corrispondente.

3.5.1 Schema generale SOP e POS



3.5.2 Equivalenza delle due forme canoniche

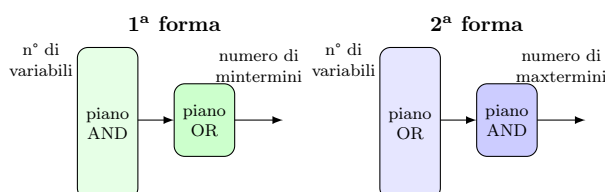
Si può dimostrare che la 1ª e la 2ª forma canonica della stessa funzione sono equivalenti, applicando le leggi di De Morgan ai mintermini negati:

$$\bar{a}bc + \bar{a}\bar{b}\bar{c} + a\bar{b}\bar{c} + abc = (a + b + c) \cdot (a + \bar{b} + \bar{c}) \cdot (\bar{a} + b + \bar{c})$$

Dimostrazione (per le leggi di De Morgan, negando due volte ogni mintermine):

$$\begin{aligned} \bar{a}bc + \bar{a}\bar{b}\bar{c} + a\bar{b}\bar{c} + abc &= \overline{\bar{a}\bar{b}\bar{c}} \cdot \overline{\bar{a}b\bar{c}} \cdot \overline{a\bar{b}c} \cdot \overline{abc} \\ &= \overline{(a + \bar{b} + \bar{c})} \cdot \overline{(a + \bar{b} + c)} \cdot \overline{(\bar{a} + b + c)} \cdot \overline{(\bar{a} + \bar{b} + \bar{c})} = (a + b + c) \cdot (a + \bar{b} + \bar{c}) \cdot (\bar{a} + b + \bar{c}) \cdots = 1 \cdot 1 \cdot 1 \end{aligned}$$

A queste due forme canoniche corrispondono due tipi di circuiti, entrambi a due piani di porte logiche e con un numero di ingressi/uscite legato al numero di variabili e al numero di mintermini (o maxtermini):



I circuiti corrispondenti alle due forme canoniche, una volta semplificati, si possono ottenere comodamente tramite le **mappe di Karnaugh**.

3.5.3 Mappe di Karnaugh a 3 variabili

Data la tabella di verità precedente, si costruisce la mappa di Karnaugh a 3 variabili $c \backslash ab$:

Se uso i maxtermini riduco l'espressione ma ho cmq tanti termini $\Rightarrow$ meglio usare i mintermini.

$c \backslash ab$	00	01	11	10
1	1	1	1	1

Se uso i mintermini trovo:
 $F(a, b, c) = b\bar{c} + a\bar{c} + ab + \bar{a}bc$

$c \backslash ab$	00	01	11	10
1	1	1	1	1

3.5.4 Esempi di mappe di Karnaugh a 4 variabili

Se uso i maxtermini riduco l'espressione ma ho cmq tanti termini $\Rightarrow$ meglio usare i mintermini.

$cd \backslash ab$	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	0

Se uso i mintermini trovo:
 $F(a, b, c, d) = bd + \bar{c}d = (b + \bar{c}) \cdot d$

$cd \backslash ab$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	1	1	0
10	0	0	0	0

3.6 Codifica binaria

Con n bit posso descrivere 2^n valori in decimale.

Esempio: $N = 13 \leftrightarrow 1101$.

Conversione binario $\rightarrow$ decimale: si moltiplica ogni cifra per la potenza di 2 corrispondente alla sua posizione e si somma:

$$1101 \rightarrow 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = 13$$

Conversione decimale $\rightarrow$ binario: si divide ripetutamente per 2, prendendo via via i resti dal basso verso l'alto:

$$13 \rightarrow \frac{13}{2} = 6 \text{ resto } 1 \rightarrow \frac{6}{2} = 3 \text{ resto } 0 \rightarrow \frac{3}{2} = 1 \text{ resto } 1 \rightarrow \frac{1}{2} = 0 \text{ resto } 1$$

$$\Rightarrow 13 = 1101 \text{ (leggendo i resti dal basso verso l'alto)}$$

3.7 Codice BCD

BCD (Binary Coded Decimal): per scrivere le cifre decimali da 0 a 9 si usano 4 bit (3 bit sarebbero troppo pochi, dato che $2^3 = 8 < 10$).

3.7.1 Esempio: display a 7 segmenti

Si vuole pilotare un singolo segmento di un display a 7 segmenti a partire dalla cifra BCD in ingresso a, b, c, d .

Si costruisce la tabella di verità del segmento (con $a \leftrightarrow$ bit più significativo, $d \leftrightarrow$ bit meno significativo) e si segnano come **don't care** ($\times$) le combinazioni che non corrispondono a nessuna cifra BCD (cioè da 10 a 15), in modo da poter scegliere il loro valore così da semplificare il circuito:

a	b	c	d	uscita
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1010 – 1111	× × × × × ×			

La 2^a forma canonica (POS) del segmento, prima della semplificazione, è:

$$F(a, b, c, d) = (a + b + c + d) \cdot (a + b + c + \bar{d}) \cdot (a + \bar{b} + \bar{c} + \bar{d})$$

Riportando la funzione (con i don't care) sulla mappa di Karnaugh a 4 variabili $cd \backslash ab$ e scegliendo opportunamente il valore dei don't care, si semplifica notevolmente il circuito:

$cd \backslash ab$	00	01	11	10
00	0	1	×	0
01	1	0	×	×
11	0	1	×	×
10	0	0	×	×

Scegliendo i don't care in modo da formare i raggruppamenti più grandi possibile, l'espressione semplificata diventa:

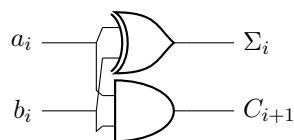
$$a + c\bar{d} + \bar{b}c + b\bar{c} = a + c\bar{d} + (b \oplus c)$$

molto più corta e immediata da realizzare rispetto alla forma canonica di partenza.

I don't care si scelgono in modo da semplificare il più possibile il circuito risultante.

3.8 Sommatore

3.8.1 Semi-sommatore (half adder)



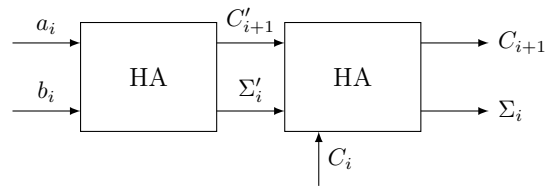
circuito di semi-sommatore (half adder)

Tabella di verità del semi-sommatore:

a_i	b_i	Σ_i	C_{i+1}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.8.2 Sommatore completo (full adder)

Il sommatore completo si ottiene combinando due semi-sommatori, in modo da tenere conto anche del riporto C_i proveniente dal semi-sommatore precedente:



FA (sommatore a 2 bit)

Tabella di verità del sommatore completo (con Σ'_i , C'_{i+1} uscite del primo HA):

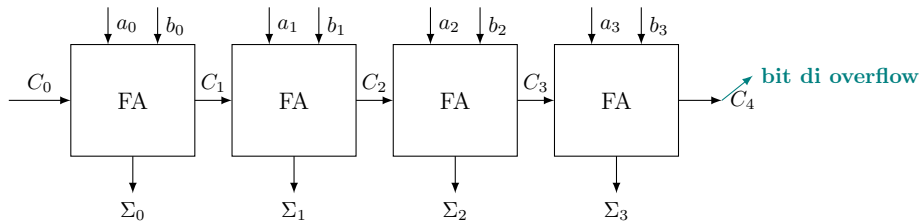
a_i	b_i	C_i	Σ'_i	Σ_i	C_{i+1}
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	1	0	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	0	0	1
1	1	1	0	1	1

Capitolo 4. Sommatori, complemento a 2 e codici

4.1 Esempio: sommatore a 4 bit

(27 marzo 2026)

Collegando in cascata 4 full adder si ottiene un sommatore a 4 bit:



Bit di overflow (C_4): se è 1, Σ_3 non è descrivibile da 4 bit $\Rightarrow$ errore.
 C_4 è il *bit in posizione maggiore*: il bit più significativo.

Il ritardo con cui si forma il carry è $3\Delta T$, con ΔT ritardo di una singola porta di un singolo sommatore; quindi il ritardo di C_4 è:

$$\Delta t = n \cdot 3\Delta T$$

Δt si risolve con il **carry look-ahead**.

4.2 Modularità

Modulare = composto da unità aggiungibili o eliminabili senza ripercussioni sul resto del sistema.

4.3 Sottrattori e numeri negativi

(27 marzo 2026 – cont.)

Per i numeri negativi c'è un bit in più per il segno: 1000 e 0000 sono entrambi 0.

Se abbiamo un numero in base r con n digit posso rappresentare al massimo $r^n - 1$ numeri. Il complemento in base r del numero N è definito come:

$$C_r(N) = r^n - N$$

quello in base $r-1$ è $C_{r-1}(N) = (r^n - 1) - N = N_{\max} - N$.

In base 2:

$$C_1(N) = 2^n - 1 - N$$

$$2^n - 1 = C_1(N) + N$$

$$C_1(N) = \overline{N}_{n-1} \cdots \overline{N}_1 \overline{N}_0$$

$$C_2(N) = 2^n - N$$

$$C_2(N) + N = C_1(N) + 1$$

$$C_2(N) = \overline{N}_{n-1} \cdots \overline{N}_1 \overline{N}_0 + 1$$

(se ho $n = 4$) $C_2(C_2(N)) = N$

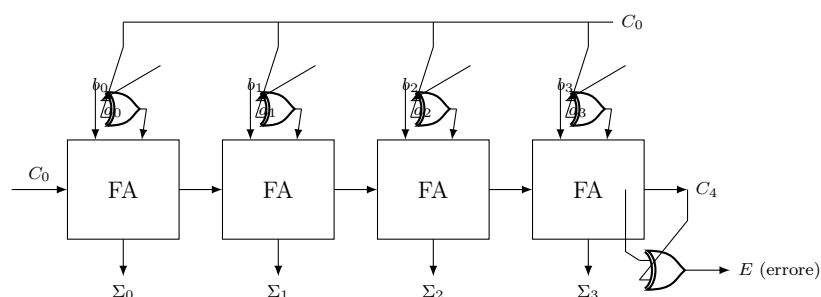
Il **complemento a 2** si usa per rappresentare i numeri negativi, considerando lo 0 e il valore $1111 \dots$:

$$+3 = 0011 \quad -3 = 1101 \quad (\text{poi } 0 \ 1111)$$

$1111 + 1 = 10000$: la rappresentazione è a 4 bit (l'1 in posizione 4 si perde perché abbiamo solo 4 digit).

4.3.1 Circuito sommatore/sottrattore a 4 bit

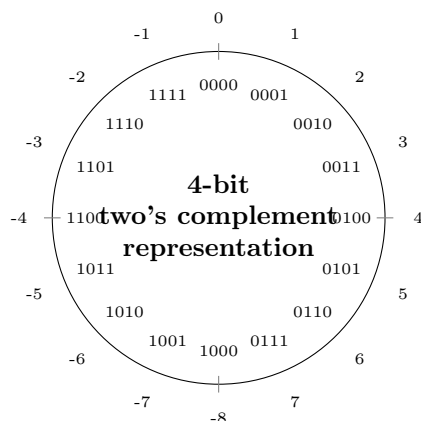
Con $a = a_3a_2a_1a_0 = 0100$ e $b = b_3b_2b_1b_0 = 0001$:



- Se $C_0 = 0$: il circuito opera da sommatore $\in [0, 15]$. *es. $b+a \rightarrow 1+4 = 5$*
- Se $C_0 = 1$: il circuito opera da sottrattore $\in [-8, 7]$. *es. $b-a \rightarrow 1-(4+1) = -4$*
- **Errore** se $E = 1$ ($C_3 \neq C_4$) come controllo di overflow.

In generale:

- Se $C_0 = 0 \Leftrightarrow a + b$.
- Se $C_0 = 1 \Leftrightarrow \bar{a} + b$, con un $+1$ sommato ad $\bar{a}$ dal suo stesso segno.
- Se $\Sigma \geq 0 \Leftrightarrow \Sigma_3 = 0$.
- Se $\Sigma < 0 \Leftrightarrow \Sigma_3 = 1$.



Capitolo 5. Codice Gray e ASCII

(30 marzo 2026)

5.1 Codice Gray

Codice Gray = rappresentazione di numeri consecutivi che differiscono per un solo digit.
La potenza dissipata da un contatore in codice Gray è sempre costante, perché ad ogni passo cambia sempre una sola cifra.

0 = 0000
1 = 0001
2 = 0011
3 = 0010
4 = 0110
5 = 0111
6 = 0101
7 = 0100

Se ho un numero con n bit:

Conversione Binario $\rightarrow$ Gray:

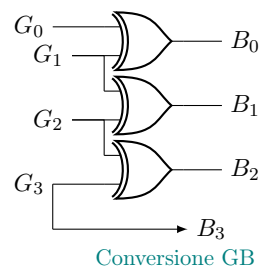
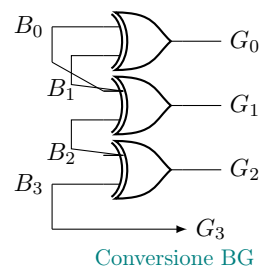
$$G_{n-1} = B_{n-1} \quad G_k = B_k \oplus B_{k+1} \quad k \in [0, n-2]$$

(G_i = cifra i -esima in Gray; B_i = cifra i -esima in Binario.)

Conversione Gray $\rightarrow$ Binario:

$$B_{n-1} = G_{n-1} \quad B_k = G_k \oplus \dots \oplus G_{n-1}$$

5.1.1 Circuiti di conversione



5.2 Codice ASCII

Codice ASCII = codice con cui si descrivono in binario, ottale ed esadecimale i simboli (lettere, cifre, caratteri di controllo).

La tabella ASCII standard a 7 bit codifica 128 simboli (2^7), tra cui lettere maiuscole (65–90), minuscole (97–122), cifre (48–57) e caratteri di controllo (0–31).

Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char
0	000	00	NUL	32	040	20	(sp)	64	100	40	@	96	140	60	`
1	001	01	SOH	33	041	21	!	65	101	41	A	97	141	61	a
2	002	02	STX	34	042	22	"	66	102	42	B	98	142	62	b
3	003	03	ETX	35	043	23	#	67	103	43	C	99	143	63	c
4	004	04	EOT	36	044	24	\$	68	104	44	D	100	144	64	d
5	005	05	ENQ	37	045	25	%	69	105	45	E	101	145	65	e
6	006	06	ACK	38	046	26	&	70	106	46	F	102	146	66	f
7	007	07	BEL	39	047	27	'	71	107	47	G	103	147	67	g
8	010	08	BS	40	050	28	(	72	110	48	H	104	150	68	h
9	011	09	HT	41	051	29	)	73	111	49	I	105	151	69	i
10	012	0A	LF	42	052	2A	*	74	112	4A	J	106	152	6A	j
11	013	0B	VT	43	053	2B	+	75	113	4B	K	107	153	6B	k
12	014	0C	FF	44	054	2C	,	76	114	4C	L	108	154	6C	l
13	015	0D	CR	45	055	2D	-	77	115	4D	M	109	155	6D	m
14	016	0E	SO	46	056	2E	.	78	116	4E	N	110	156	6E	n
15	017	0F	SI	47	057	2F	/	79	117	4F	O	111	157	6F	o
16	020	10	DLE	48	060	30	0	80	120	50	P	112	160	70	p
17	021	11	DC1	49	061	31	1	81	121	51	Q	113	161	71	q
18	022	12	DC2	50	062	32	2	82	122	52	R	114	162	72	r
19	023	13	DC3	51	063	33	3	83	123	53	S	115	163	73	s
20	024	14	DC4	52	064	34	4	84	124	54	T	116	164	74	t
21	025	15	NAK	53	065	35	5	85	125	55	U	117	165	75	u
22	026	16	SYN	54	066	36	6	86	126	56	V	118	166	76	v
23	027	17	ETB	55	067	37	7	87	127	57	W	119	167	77	w
24	030	18	CAN	56	070	38	8	88	130	58	X	120	170	78	x
25	031	19	EM	57	071	39	9	89	131	59	Y	121	171	79	y
26	032	1A	SUB	58	072	3A	:	90	132	5A	Z	122	172	7A	z
27	033	1B	ESC	59	073	3B	;	91	133	5B	[	123	173	7B	{
28	034	1C	FS	60	074	3C	<	92	134	5C	\	124	174	7C	
29	035	1D	GS	61	075	3D	=	93	135	5D	]	125	175	7D	}
30	036	1E	RS	62	076	3E	>	94	136	5E	^	126	176	7E	~
31	037	1F	US	63	077	3F	?	95	137	5F	_	127	177	7F	DEL

Legenda: le colonne da sinistra a destra mostrano decimale, ottale, esadecimale e il carattere corrispondente. I caratteri di controllo (0–31 e 127) sono abbreviati (es. NUL=null, SOH=start of heading, CR=carriage return, DEL=delete, ecc.).

Intervalli notevoli:

- Caratteri di controllo: 0–31 e 127;
- Spazio: 32;
- Caratteri stampabili: 33–126;
- Cifre: 48–57 (0–9);
- Lettere maiuscole: 65–90 (A–Z);
- Lettere minuscole: 97–122 (a–z).

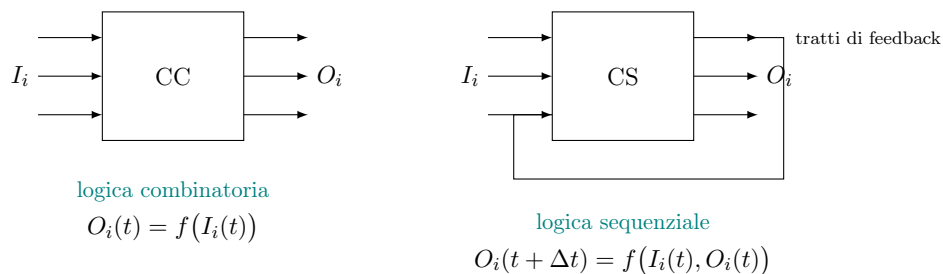
Capitolo 6. Logica sequenziale

6.1 Circuiti combinatori e circuiti sequenziali

Finora abbiamo visto circuiti in cui l'output dipende esclusivamente dagli input: **circuiti combinatori**.

Esistono anche circuiti in cui lo stato delle uscite dipende anche dallo stato in cui il circuito era in precedenza: questi circuiti seguono una **logica sequenziale**.

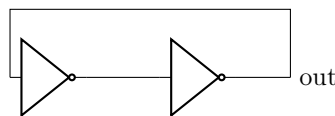
Nei circuiti sequenziali è necessario che ci sia una **memoria**, un feedback.



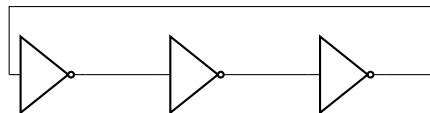
6.2 Bistabilità e oscillazione

Il **circuito minimale** che mantiene uno stato stabile è il **bistabile**: rinforza il suo stato ad ogni loop.

Il problema è che non si sa come invertirlo $\Rightarrow$ si interviene con porte **NAND** o **NOR**.



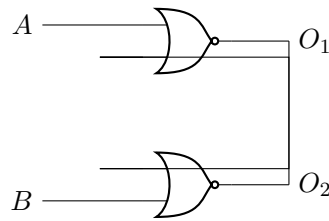
porte NOT pari $\rightarrow$ circuito stabile (bi-stabile)



porte NOT dispari $\rightarrow$ circuito oscillante ($T \propto 2n+1$)

6.3 Latch

Latch = circuito base dell'elettronica digitale sequenziale.



LATCH (NOR)

Tabella di verità del latch NOR (con O_1, O_2 a seconda dello stato precedente, indicato come O_1^{old}/O_2^{old}):

A	B	O_1	O_2
0	0	O_1^{old}	O_2^{old}
0	1	0	1
1	0	1	0
1	1	0	0

- Se $A = 0, B = 0$: porta **HOLD** (mantiene lo stato precedente);
- Se $A = 0, B = 1$: porta **NOR**;
- Se $A = 1, B = 1$: **stato proibito**.

Nota: c'è dipendenza dalla velocità delle porte.

Ci sono problemi all'uscita del circuito:

1. Se le porte hanno ritardi diversi, le uscite sono 1 e 0, ma l'output dipende dalle singole componenti.
2. Se le porte hanno ritardi uguali, le uscite sono 1,0 oppure alternanti, quindi è un'oscillazione con $T = \text{ritardo}$.

$$O_1 = \overline{B(t) \cdot \overline{O_2(t)} \cdot \overline{O_2(t + \Delta t)}}$$

6.3.1 Latch RS con NOR: funzione caratteristica

Analizziamo la **funzione di trasferimento** del latch RS realizzato con porte NOR. Partendo dall'uscita della prima porta:

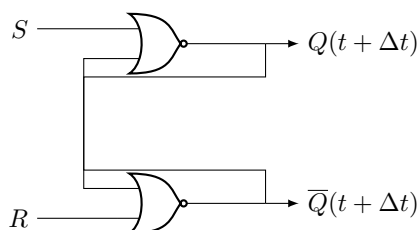
$$Q(t + \Delta t) = \overline{R + (S + Q(t))} = \overline{R}S + \overline{R}Q(t) + \overline{RS}$$

Il termine $\overline{RS}$ rappresenta lo *stato proibito* (circolo evidenziato negli appunti). Poiché $RS = 1$ non può mai verificarsi in condizioni normali, si può scrivere la **funzione caratteristica del latch RS con NOR**:

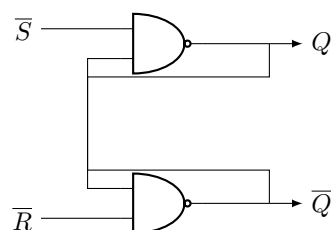
$$Q(t + \Delta t) = S + \overline{R}Q(t)$$

R	S	$Q(t + \Delta t)$	$\overline{Q}(t + \Delta t)$	
0	0	$Q(t)$	$\overline{Q}(t)$	HOLD
0	1	1	0	SET
1	0	0	1	RESET
1	1	0	0	(PROIBITO)

Il circuito corrispondente (due NAND con feedback incrociato, ingressi $Q(t)$, S e R) è mostrato di seguito insieme alla variante con porte NAND:



Latch SR con porte NOR

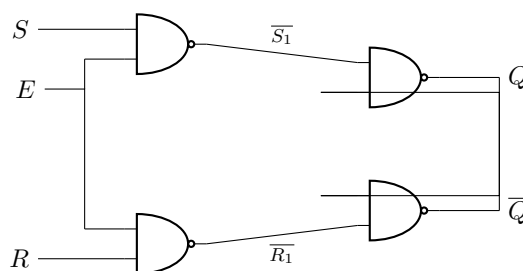


Latch SR con porte NAND

Nel latch con porte **NAND**, la stessa tabella vale scegliendo $\bar{R}$ e $\bar{S}$ come ingressi al posto di R e S .

6.3.2 Latch RS con abilitazione (Enable)

I circuiti visti finora non hanno un'**abilitazione**: non c'è modo di rendere il circuito insensibile agli ingressi. Si introduce il segnale E (Enable) mediante due porte NAND aggiuntive ($\bar{S}_1$ e $\bar{R}_1$):

Latch di tipo RS (con abilitazione E)

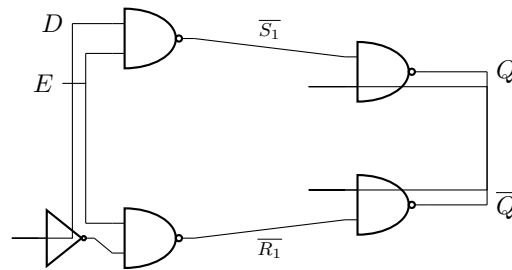
- Se $E = 0$: $SE = RE = 0 \Rightarrow \bar{S}_1 = \bar{R}_1 = 1 \Rightarrow$ **HOLD** (circuito insensibile agli ingressi).
- Se $E = 1$: $\bar{S}_1 = \bar{S}$ e $\bar{R}_1 = \bar{R} \Rightarrow$ circuito **abilitato**. Rimane però il problema dello **stato proibito** ($R = S = 1$).

Funzione caratteristica del latch RS con abilitazione:

$$Q(t + \Delta t) = ES + \bar{R}\bar{E}Q(t)$$

6.3.3 Latch di tipo D

Per eliminare lo stato proibito si inserisce un **invertitore** tra S e R , in modo che $R = \bar{D}$ sia sempre il complemento di $S = D$: nasce il **latch D** (Data).



Latch di tipo D

Funzione di trasferimento del latch D:

$$Q(t + \Delta t) = DE + DQ(t) + \overline{E}Q(t)$$

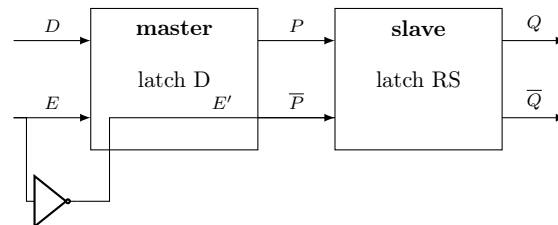
6.3.4 Latch e Flip-Flop: differenza fondamentale

- **Latch** = sensibile a un *livello* del segnale; segnale di controllo: E (Enable).
- **Flip-Flop** = sensibile a un *fronte* del segnale; segnale di controllo: C (Clock).

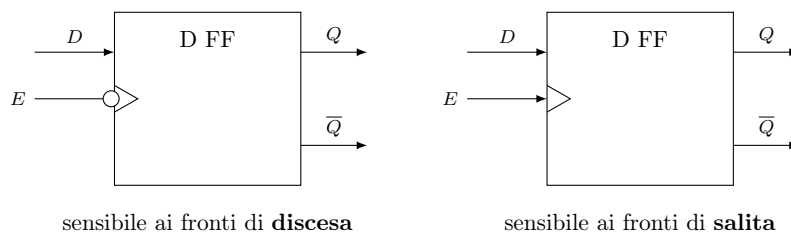
Convenzione simbolica: se non si mette il simbolo $\triangleright$, ci si riferisce al livello; se lo si mette, ci si riferisce al fronte.

6.3.5 Tecnica master-slave

Per realizzare un flip-flop sensibile al fronte si cascadiano due latch: il **master** (latch D) riceve E , lo **slave** (latch RS) riceve $E' = \overline{E}$ attraverso un invertitore. Le uscite intermedie del master sono P e $\overline{P}$.



I due simboli equivalenti del flip-flop master-slave (a seconda del fronte attivo):

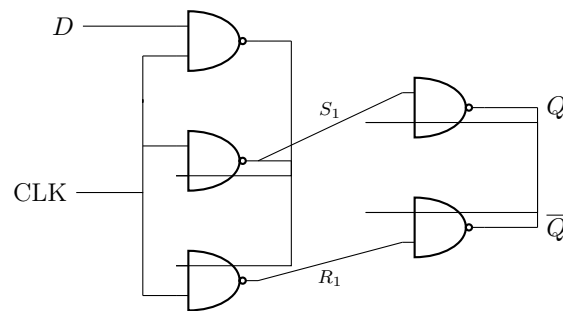


Comportamento del sistema master-slave:

- $E = 1 \Rightarrow E' = 0 \Rightarrow Q = \text{cte}$ (slave bloccato; master segue D).
- $E = 0 \Rightarrow E' = 1, S = \overline{R} = P \Rightarrow Q = \text{cte}$ (master bloccato; slave propaga P).
- $E \uparrow \Rightarrow E' \downarrow \Rightarrow Q = \text{cte}$.
- $E \downarrow \Rightarrow E' \uparrow \Rightarrow Q$ cambia e diventa D .

6.4 Tecnica trigger-edge

La **tecnica trigger-edge** è un'alternativa al master-slave: il flip-flop cambia stato solo in corrispondenza di un preciso fronte del clock, grazie ad una rete di cinque porte NAND.



Flip-Flop D trigger-edge (5 porte NAND)

- Sul **fronte di discesa** del CLK ($1 \rightarrow 0$): Q acquisisce il valore di D e $\bar{Q}$ quello di $\bar{D}$.
- Se D cambia mentre CLK è a regime (livello stabile), l'uscita **resta costante**.

Riferimento laboratorio: pag. 9–10–11–12 di `digitale/lab3-09-2026o410-latchFF.pdf`.

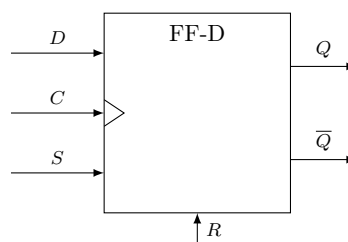
6.5 Ingressi sincroni e asincroni

(13/04/2026)

Funzionamento sincrono = i cambi di stato delle uscite avvengono sul *fronte* del clock.

Funzionamento asincrono = le uscite *non* dipendono dal valore del clock.

Nel flip-flop D con ingressi misti (tipico delle logiche TTL):

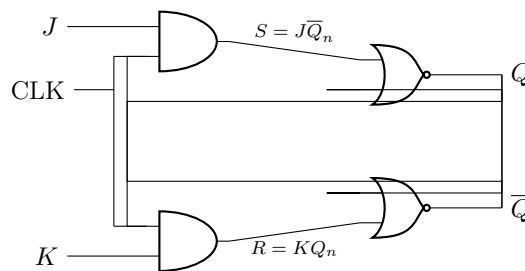


FF-D con ingressi misti

- D è **sincrono** al clock.
- R e S sono **asincroni** in logica TTL (agiscono indipendentemente dal clock).

6.6 JK Flip-Flop

Il **JK flip-flop** è ottenuto da un latch RS aggiungendo due porte AND che moltiplicano gli ingressi J e K per i feedback di $\bar{Q}$ e Q rispettivamente: $S = J\bar{Q}_n$ e $R = KQ_n$.



JK Flip-Flop

Funzione caratteristica:

$$Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$$

Dalla funzione di trasferimento del latch SR con NOR ($Q_{n+1} = S + \bar{R}Q_n$), ponendo $S = J\bar{Q}_n$ e $R = KQ_n$:

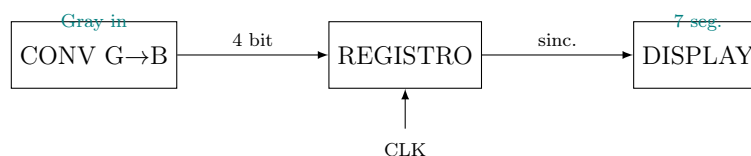
$$Q(t + \Delta t) = S + R\bar{Q}(t) \Rightarrow Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$$

Casi notevoli:

- $J \neq K$: **SET** ($J = 1, K = 0$) e **RESET** ($J = 0, K = 1$).
- $J = K = 0$: **HOLD** ($Q_{n+1} = Q_n$).
- $J = K = 1$: **TOGGLE** ($Q_{n+1} = \bar{Q}_n$).

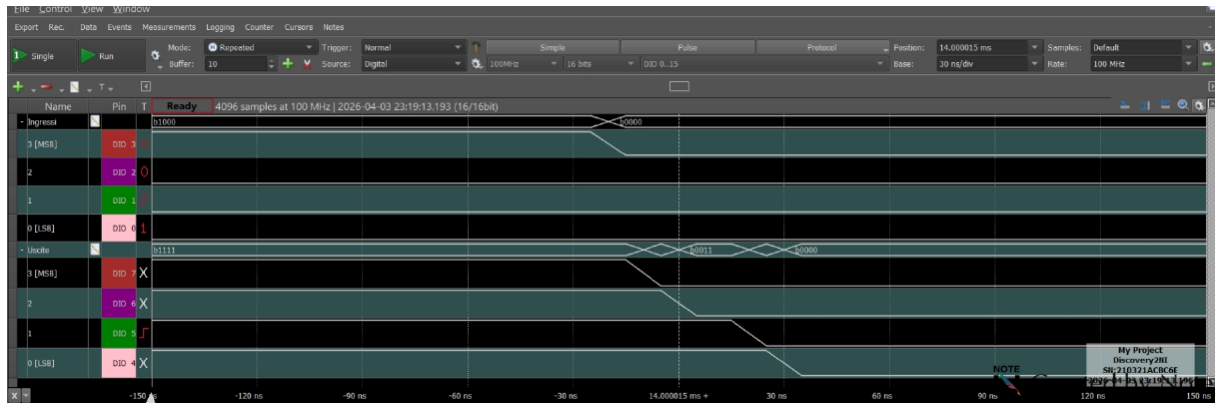
6.6.1 Esempio: registro tra convertitore Gray-Binario e display 7 segmenti

Si consideri un sistema composto da un **convertitore Gray-Binario** e un **display a 7 segmenti**. Durante la transizione da 15 a 0 (passaggio b1000 $\rightarrow$ b0000 in ingresso Gray), le uscite binarie percorrono stati intermedi che causano glitch visibili sul display (le uscite passano per b1111 $\rightarrow$ b0011 $\rightarrow$ b0000 con transizioni disallineate nei singoli bit).



Inserendo un **registro** (banco di flip-flop sincronizzati dallo stesso clock) tra il convertitore e il display, le uscite vengono bloccate e aggiornano il display *solo* sul fronte del clock, eliminando i glitch. Come mostra la misura all'analizzatore logico (Digilent Discovery 2NI, 100 MHz, 30 ns/div):

- **Ingressi** (Gray): transizione netta da b1000 a b0000 (cambio di un solo bit alla volta).
- **Uscite senza registro**: la transizione da b1111 a b0000 passa per stati intermedi (b0011 ecc.) con i singoli bit che commutano in tempi diversi (glitch).
- **Uscite con registro**: il display vede le uscite sincronizzarsi al fronte del clock, eliminando le transizioni spurie.



Le uscite (gruppo *Uscite*) mostrano le transizioni disallineate dei singoli bit: da b1111 passando per b0011 fino a b0000, con i quattro bit che commutano in istanti diversi (glitch visibile nella zona a cavallo di $t = 14,000015 \text{ ms}$). Il registro sincronizzato elimina questi glitch, aggiornando il display solo sul fronte del clock.

Capitolo 7. Registri a scorrimento e contatori

(17 aprile 2026)

7.1 Registri a scorrimento

I **registri a scorrimento** agiscono come **passatori** fra due stati del circuito:

$$Q_0 = X \rightarrow Q_1 = X, \quad Q_0 = Y \rightarrow Q_2 = X, \quad Q_1 = Y, \quad Q_0 = Z \rightarrow \dots$$

7.1.1 Modalità di trasmissione dati

I registri a scorrimento hanno molteplici utilizzi nella trasmissione dati. (*Nota: non è a scorrimento.*)

- **Seriale** $\rightarrow$ **seriale** = **SISO**: $\text{in} \rightarrow [D_2 \rightarrow D_1 \rightarrow D_0] \rightarrow \text{out}$ *usato per il delaying*
- **Seriale** $\rightarrow$ **parallelo** = **SIPO**: $\text{in} \rightarrow [D_2 \rightarrow D_1 \rightarrow D_0] \rightarrow \text{out}$ *usato per il filling*
- **Parallelo** $\rightarrow$ **seriale** = **PISO**: $\text{in} \rightarrow [D_2, D_1, D_0] \rightarrow \text{out}$ *usato per il drawing*
- **Parallelo** $\rightarrow$ **parallelo** = **PIPO**: $\text{in} \rightarrow [D_2, D_1, D_0] \rightarrow \text{out}$ *usato per lo storing*

I tempi di flip-flop sono sufficientemente brevi, rispettati anche dalla frequenza del clock.

7.1.2 Temporizzazione e vincoli

$$T_P > T_H, \quad T_S + T_P < T_{CLK}$$

- T_{setup} e T_{hold} : tempi caratteristici del flip-flop rispetto al fronte di clock.
- $I_N = D_{SR}$: dato seriale in ingresso.
- $Q_0 = D_1$: il primo flip-flop segue l'ingresso.

Nei registri sono presenti alcuni controlli:

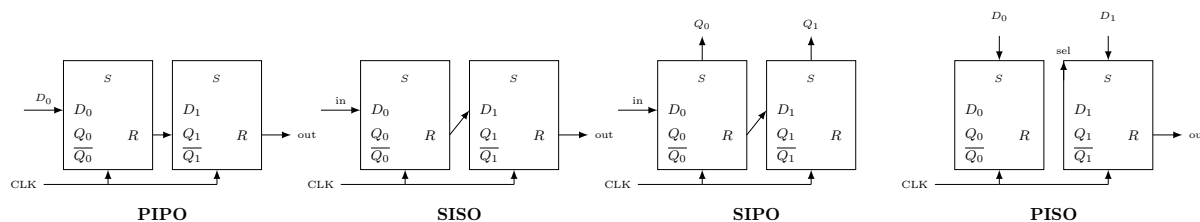
- **clock** = sincronizza lo shift;
- **reset/clear** = azzerà tutto;
- **set** = una segnale;
- **enable** = abilita lo shift.

Si distinguono due tipi di registro:

- **Registri normali (PIPO)**: conservano i dati;
- **Registri a scorrimento**: conservano e spostano i dati.

7.1.3 Schemi dei registri a scorrimento (SISO, SIPO, PISO)

Di seguito sono riportati gli schemi a blocchi semplificati per le tre configurazioni più comuni (ogni blocco è un FF-D con uscite Q_0 , $\overline{Q_0}$, ingresso D e clock R/CLK).

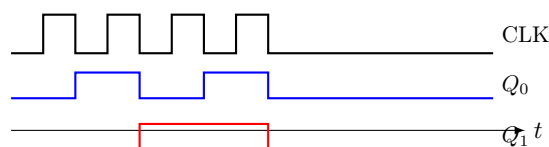


7.2 Contatori

(17 aprile 2026)

Contatore = circuito che esegue le transizioni da uno stato al successivo, rappresentando in binario numeri contigui.

- Q_0 cambia ogni periodo (del clock).
- Q_1 cambia ogni 2 periodi (quando Q_0 ha fronte di discesa).
- Q_{i+1} cambia ogni 2^{i+1} periodi (quando Q_i ha fronte di discesa).

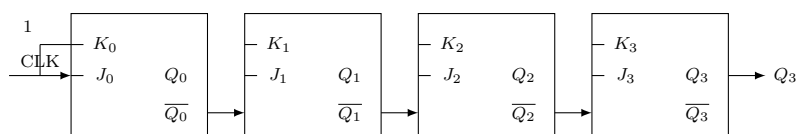


7.2.1 Contatore asincrono (ripple counter)

Non tutte le parti del circuito sono guidate dallo stesso clock: **il contatore è asincrono**. Il clock di ciascun blocco è legato all'uscita del precedente.

Il tempo di propagazione del bit più significativo è $T_n = n T_P$.

Deve valere $T_{CLK} > T_n$ per avere un tempo in cui il segnale è stabile.



CONTATORE ASINCRONO

7.2.2 Contatore sincrono

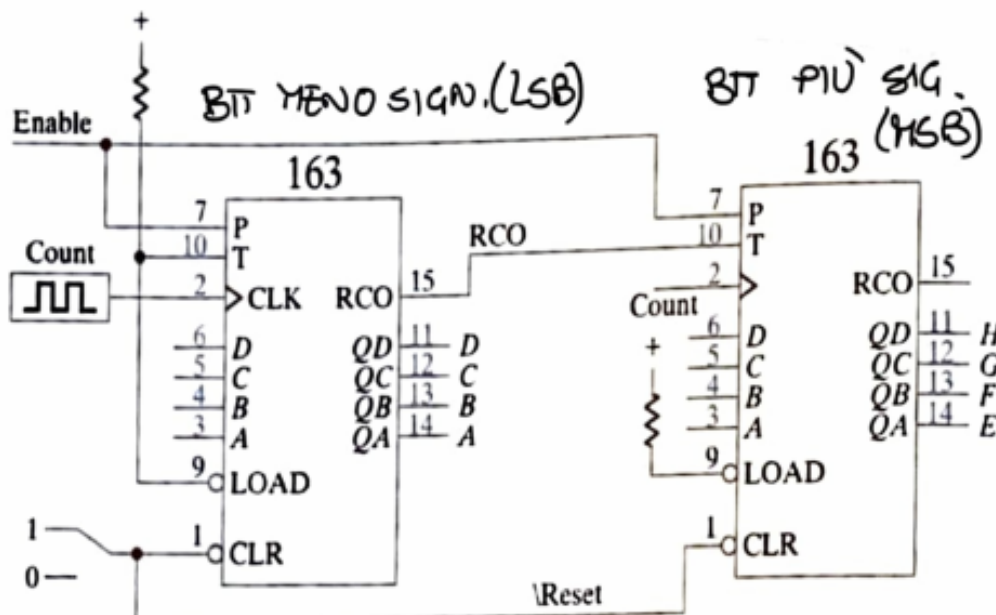
Tutte le parti sono guidate dallo stesso clock: ogni uscita dei blocchi è collegata all'uscita del blocco precedente con una porta AND, che fa cambiare stato al successivo *solo se tutte le uscite prima sono sature* (cioè tutte a 1).

Capitolo 8. Conteggio, divisori di frequenza e PRFG

(20 aprile 2026)

8.1 Contatori a 8 bit: cascata di due SN74LS163

Due contatori sincroni a 4 bit si collegano in **cascata** tramite il segnale RCO: il bit più significativo (*MSB*, gestito dal secondo 163) viene incrementato quando il contatore meno significativo (*LSB*) raggiunge il valore 15 e porta $RCO = 1$. Il segnale RCO del primo modulo funge da *enable di conteggio* per il secondo.



Due contatori sincroni a 4 bit si collegano a cascata con il RCO che funge da *enable di conteggio*: il contatore MSB avanza di un'unità ogni volta che il LSB arriva a 15 ($RCO = 1$). In questo modo si ottiene un contatore a 8 bit capace di rappresentare i valori da 0 a 255 ($2^8 - 1$). Le uscite del secondo modulo (*H, G, F, E*) sono i bit più significativi dell'intero conteggio; il segnale $\overline{Reset}$ asincrono è comune ad entrambi i moduli.

8.2 Divisori di frequenza per un numero n

(20 aprile 2026)

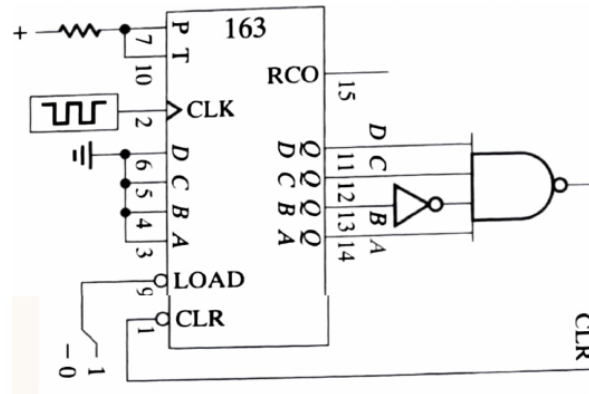
Divisore di frequenza = circuito sequenziale che prende in ingresso il segnale di clock e ne riduce la frequenza di un fattore n :

$$f_{\text{out}} = \frac{f_{\text{CLK}}}{n} \quad T_{\text{out}} = n T_{\text{CLK}}$$

Si realizzano con il contatore SN74LS163 sfruttando il segnale RCO o il LOAD. Esistono due varianti principali.

8.2.1 Divisore con CLEAR sincrono

Il circuito conta fino a DCBA = 1101 (13); dopodiché il CLEAR è 1 e ricomincia da capo.

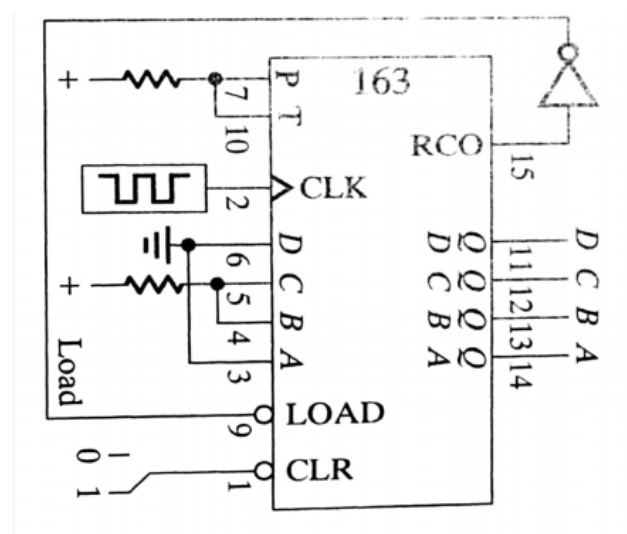


- A (LSB) ha $T_A = 2T_{CLK}$.
- D (MSB) ha $T_D = 16T_{CLK}$.
- Il numero 13 è l'ultimo contato, con $f = f_{CLK}/14$.
- Senza il clear sincrono l'ultimo stato (13) dura poco, non è esattamente un fronte.

Funzionamento: le uscite $Q_DQ_CQ_BQ_A$ vengono decodificate da una porta AND (con buffer invertente sul segnale C per selezionare il pattern 1101); quando l'AND vale 1, il segnale CLR si attiva e azzerà il contatore al colpo di clock successivo. L'uscita CLR torna quindi a 0 e il ciclo ricomincia.

8.2.2 Divisore con SET sincrono (LOAD)

Il circuito ha il RCO collegato al LOAD (set) in modo che, dopo aver finito di contare, venga caricato il valore impostato su DCBA = 0110 (6).



- RCO ha una frequenza di 4/ciclo T_{CLK} (1/10).

- In generale: $T_s = (2^n - n_{\text{load}}) T_{\text{CLK}}$.

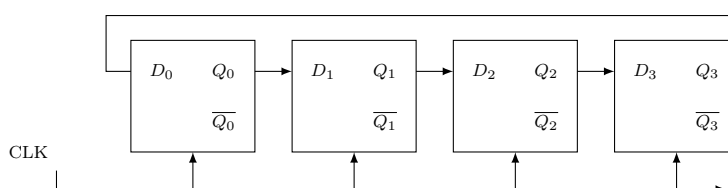
Funzionamento: quando il contatore raggiunge 15 e $\text{RCO} = 1$, al colpo di clock successivo il LOAD forza il caricamento del valore preset ($\text{DCBA} = 0110$ in questo esempio); il contatore riparte da 6 anziché da 0, ottenendo un ciclo di lunghezza $16 - 6 = 10$, cioè $f_{\text{out}} = f_{\text{CLK}}/10$. Il LED collegato all'RCO si accende una volta per ciclo.

8.3 Contatore ad anello

Si pone in ingresso $Q = 0001$ e si applica il clock per vedere come evolve:

$$0001 \rightarrow 0010 \rightarrow 0100 \rightarrow 1000$$

I FF copiano il valore in entrata: contatore con solo **4 valori**.

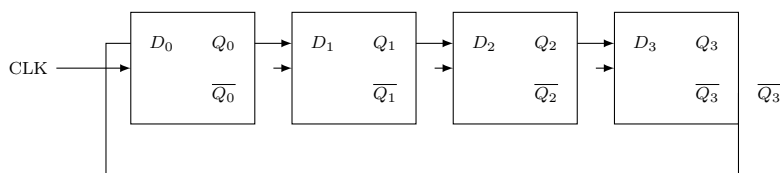


8.4 Contatore ad anello *twisted* (Johnson counter)

Si pone in ingresso $Q = 0000$ e si applica il clock; si nota che $\overline{Q_3}$ è collegato a D_0 (feedback invertito):

$$0000 \rightarrow 0001 \rightarrow 0011 \rightarrow 0111 \rightarrow 1111 \rightarrow 1110 \rightarrow 1100 \rightarrow 1000 \rightarrow 0000$$

I FF copiano il valore in entrata: contatore con **8 valori** (con 4 FF).



Rispetto al contatore ad anello normale, il Johnson counter ha **2N stati** per N flip-flop utilizzati per realizzarlo.

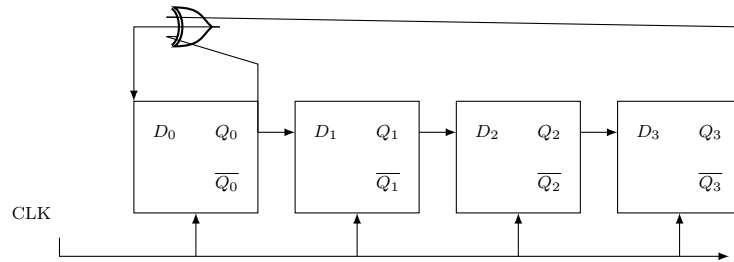
8.5 Generatori di frequenza pseudo casuali (PRFG)

PRFG (*Pseudo-Random Frequency Generator*) = applicazione dei registri a scorrimento che ha trovato grande applicazione nella generazione di sequenze di bit pseudo casuali. Il registro ha grandezza fissa; questo comporta lo “pseudo”.

Un registro largo K FF genera una sequenza ripetuta al massimo dopo $2^K - 1$ eventi.

Nei generatori pseudo casuali **lineari** si usa lo XOR (o XNOR) per costruire il circuito: le porte equivalenti alla somma binaria.

LPRG = *Linear Pseudo Random Generator*



La tabella degli stati del LPRG a 3 FF (FF0, FF1, FF2), con feedback XOR tra FF2 e FF1, mostra la sequenza pseudo casuale di lunghezza $2^3 - 1 = 7$:

Clock	FF0	FF1	FF2
0	1	1	1
1	0	1	1
2	1	0	1
3	1	1	0
4	0	1	1
5	1	0	1
6	0	0	1
7	1	0	0
8	1	1	0
9	0	1	1
10	0	0	1
11	1	0	0
12	1	1	0
13	1	1	1

Capitolo 9. Macchine a stati finiti (FSM)

(24 aprile 2026)

FSM (*Finite State Machine*) = architettura in cui gli stati presenti determinano quelli futuri.

Diagramma di stato = mappa per rappresentare il comportamento di un sistema sequenziale (possibili stati e legami).

Gli stati sono legati fra loro da precise transizioni. Gli stati sono definiti da condizioni **INTERNE** (presenti nel sistema) e/o da condizioni **ESTERNE** (segnali in ingresso dall'esterno).

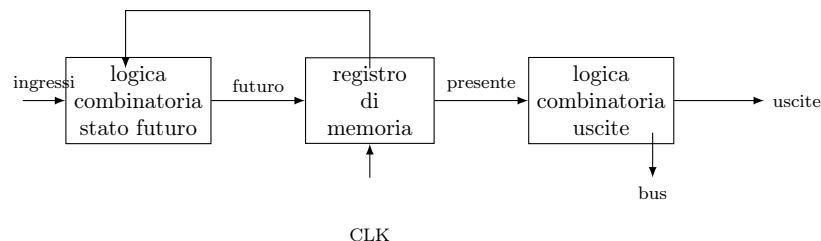
Ogni circuito digitale è una FSM.

9.1 Tipi di architettura FSM: Moore e Mealy

Esistono due tipi principali di architettura per le FSM:

- Architettura di Moore
- Architettura di Mealy

9.1.1 Architettura di Moore

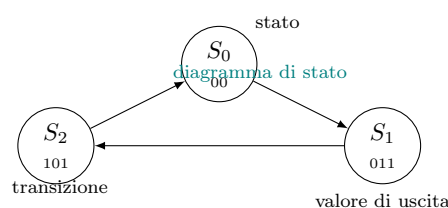


- Uno stato è univocamente conseguenza dello stato precedente.
- Gli ingressi esterni non sempre sono presenti.

9.2 Dal diagramma di stato al circuito

(24 aprile 2026)

Il diagramma di stato è composto da **stati** (cerchi etichettati con il valore di uscita) collegati da **transizioni** (freccie etichettate con il valore dell'ingresso/condizione):



La procedura per passare dal diagramma di stato al circuito è la seguente:

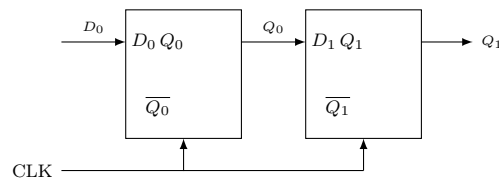
1. **Dato il numero di stati** (3) definisco il numero di bit che mi serve a rappresentarli:
 $\lceil \log_2 3 \rceil = 2 \Rightarrow 2 \text{ bit}.$
2. **Associo a ogni stato una rappresentazione in bit:** $S_0 = [00]$, $S_1 = [01]$, $S_2 = [10]$,
 $S_3 = [X]$ (*don't care*).
3. **Per ogni bit uso un FF:** $S_i = Q_i Q_0^i.$
4. **Scrivo la tabella di verità:** associo a ogni stato presente uno stato futuro.

Q_1	Q_0	Q_1^+	Q_0^+
0	0	0	1
0	1	1	0
1	0	0	0
1	1	$\times$	$\times$

5. **Scelgo il tipo di FF** (D, JK, T): in questo caso uso DFF, poiché $Q^+ = D.$
6. **Ricavo le funzioni di $D_i = f(Q_i)$:**

$$D_1 = Q_0 \quad D_0 = \overline{Q_1} \cdot \overline{Q_0}$$

7. **Disegno il circuito:**



- **DFF:** $Q^+ = D$ più usato, semplice.
- **TFF:** $Q^+ = T \oplus Q$ per contatori e sequenze regolari.
- **JKFF:** flessibile, può fare tutto (set, reset, toggle).

9.2.1 Ricavare le funzioni dei segnali di uscita: $O_i = f(Q_i)$

Q_1	Q_0	D_0	D_1	O_2	O_1	O_0
0	0	1	0	0	1	0
0	1	0	1	0	1	1
1	0	0	0	1	0	1
1	1	$\times$	$\times$	$\times$	$\times$	$\times$

$$O_0 = Q_0 + Q_1 \quad O_1 = \overline{Q_1} \quad O_2 = Q_1$$

9.2.2 Stato bloccante e segnale di Enable

Al punto 9 si verifica quale sia lo stato futuro del caso $Q_1 = Q_0 = 1$: se è uno **stato bloccante**, è un problema che dobbiamo risolvere.

Nel nostro caso, da $Q = 11$ si andrà a finire in $D = 10$, quindi ricomincerà la sequenza (non blocca il circuito).

Se volessimo scorrere il ciclo al contrario, possiamo inserire un segnale di **Enable**:

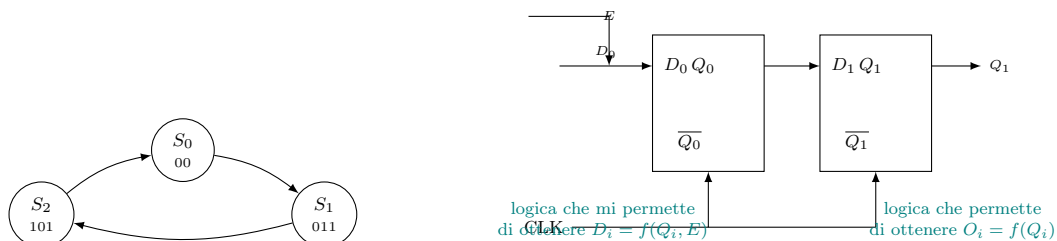
- $E = 1$: si ha $010 \rightarrow 011 \rightarrow 101$ (dritto).
- $E = 0$: si ha $101 \rightarrow 011 \rightarrow 010$ (rovescio).

Cambiamo le entrate D_i ma le uscite restano le stesse:

$$D_i = f(Q_i, E) \quad O_i = f(Q_i)$$

Possiamo scrivere le funzioni:

$$D_1 = \overline{E} \cdot \overline{Q_1} \cdot \overline{Q_0} + E \cdot Q_0 \quad D_0 = E \cdot \overline{Q_1} \cdot \overline{Q_0} + \overline{E} \cdot Q_1$$



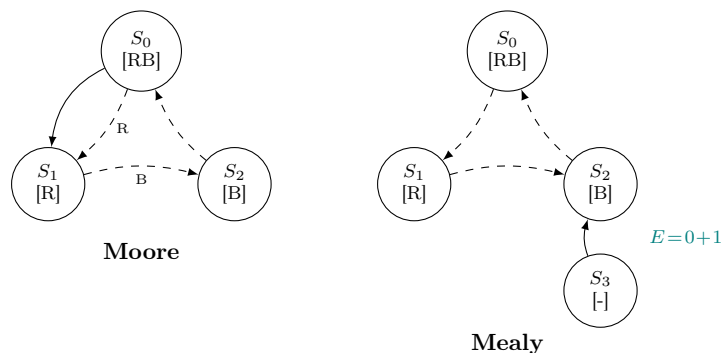
9.2.3 Esempio: insegna luminosa (continuazione)

La sequenza dell'insegna luminosa, con $E = 1$, è:

$$RABA \rightarrow RABS \rightarrow RSBA \rightarrow RABA \rightarrow \dots \quad (E = 1)$$

$$BA \rightarrow BS \rightarrow BA \rightarrow \dots \quad (E = 0, RS \text{ costante})$$

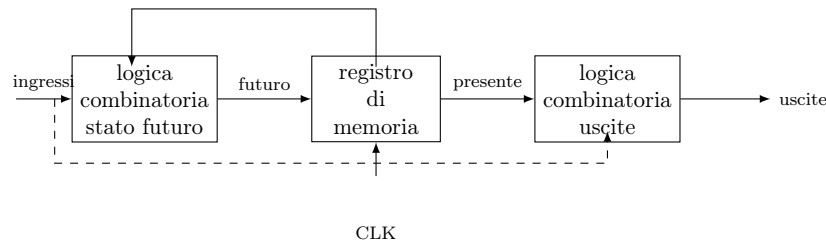
Per aggiungere lo stato “ RS costante” disabilitato, dobbiamo aggiungere un 4° stato, perché $O_i \leftrightarrow f(Q_i) \not\equiv E$.



Possiamo quindi implementare l'architettura di Moore.

9.3 Architettura di Mealy

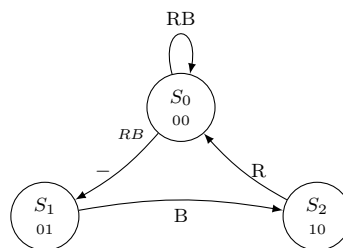
(27 aprile 2026)



La differenza principale fra Moore e Mealy è la dipendenza delle uscite dai parametri del circuito:

- in **Moore**: $O_i = f(Q_i)$, sincrona al clock.
- in **Mealy**: $O_i = f(Q_i, I_i)$, asincrona.

Il valore delle uscite si associa alle **transizioni**. Riprendendo l'esempio dell'insegna luminosa, il diagramma di Mealy è:



- $E = 1$: l'uscita è legata al passaggio fra 2 stati, non allo stato di per sé.
- $E = 0$: il fronte di clock è asincrono, perché dipende anche da E , che è asincrono: se il clock si ferma ma E cambia, allora le uscite cambiano.

Per implementare il circuito dei registri si usano JKFF in modalità “**toggle**” (1) oppure “**hold**” (0): $J = K$ in una sola modalità.

9.3.1 Tabella di verità dell'insegna luminosa (Mealy)

		stato futuro		uscite	
Q_1	Q_0	Q_1^+	Q_0^+	R	B
$E = 1$					
0	0	0	1	1	1
0	1	1	0	1	0
1	0	0	0	0	1
1	1	×	×	×	×
$E = 0$					
0	0	0	0	1	0
0	1	0	1	0	1
1	0	0	1	0	1
1	1	×	×	×	×

9.3.2 Tabella JK per l'implementazione

Usando JKFF con $J = K$ (modalità toggle quando $Q_i \neq Q_i^+$, hold quando $Q_i = Q_i^+$):

Q_1	Q_0	Q_1^+	Q_0^+	JK_1	JK_0
$E = 1$					
0	0	0	1	0	1
0	1	1	0	1	1
1	0	0	0	1	0
1	1	×	×	×	×
$E = 0$					
0	0	0	0	0	0
0	1	0	1	0	0
1	0	0	1	1	1
1	1	×	×	×	×

- Quando $Q_i = Q_i^+$: fa **hold** ($JK = 0$).
- Quando $\overline{Q_i} = Q_i^+$: fa **toggle** ($JK = 1$).

$$Q_1^+ = E Q_0 \quad Q_0^+ = \overline{Q_0} (\overline{E} + \overline{Q_1})$$

9.3.3 Mappe di Karnaugh per JK_0 e JK_1

Mappa JK_0					Mappa JK_1				
$E \backslash Q_1 Q_0$	00	01	11	10	$E \backslash Q_1 Q_0$	00	01	11	10
0	0	0	×	0	0	0	0	×	1
1	1	1	×	0	1	0	1	×	1

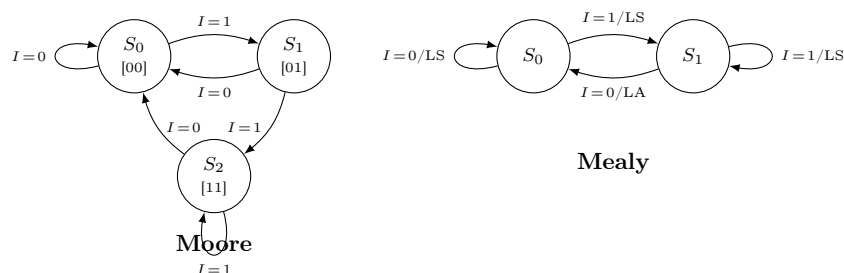
$JK_0 = \overline{Q_1} + \overline{E}$ $JK_1 = Q_1 + E Q_0$

9.4 Esempio: riconoscitore di sequenze

In ingresso c'è una serie di bit: vogliamo accendere un LED quando trovo $Q_{i+1}Q_i = 11$:

Serie di bit: $I = 10110101110001 \dots$ **LED acceso**

- S_0 = non ho ancora trovato il 1° 1: LED spento (S).
- S_1 = ho trovato il 1° 1: LED spento (S).
- S_2 = ho trovato il 2° 1: LED acceso (A).
- $I = 1, I = 0$.



9.5 Esempio: macchina distributrice del caffè

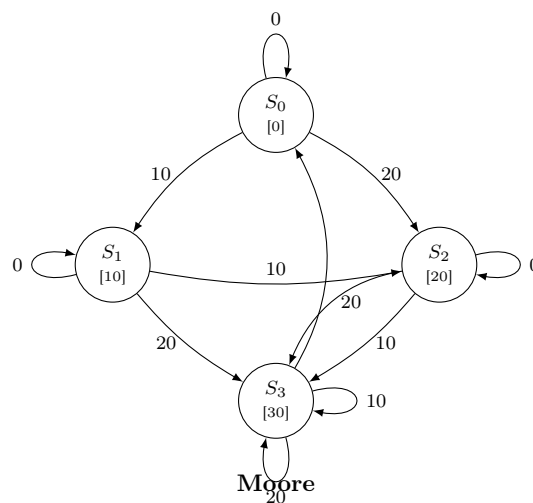
- Costo caffè: 0,30 €.
- L'erogazione del caffè avviene se metto $\geq 0,30$ €.
- La macchina **non dà resto**.
- La macchina riconosce il peso e accetta solo monete da 0,10 € e 0,20 €.

Stati della macchina (2 FF, 4 stati):

Stato	Credito
S_0	0 cent
S_1	10 cent
S_2	20 cent
S_3	≥ 30 cent

9.5.1 Diagramma di stato della macchina del caffè (Moore)

Per Mealy ogni stato non deve avere uscite mai definite, altrimenti ci sono dei problemi di confusione.



Capitolo 10. Elettronica combinatoria complessa

(11 maggio 2026)

Circuiti che permettono di implementare funzioni in modo compatto.

10.1 ROM – Read Only Memory

ROM (*Read Only Memory*): può essere interpretata in 2 modi:

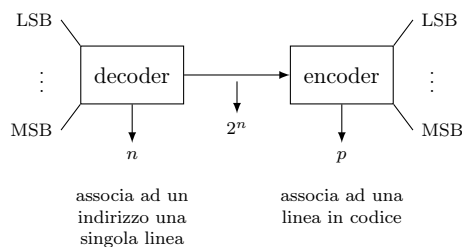
1. **Memoria** da 2^n parole da p bit.
2. **Implementazione** di p funzioni delle n variabili in ingresso.

Per n linee di ingresso ho 2^n possibili combinazioni; p sono le linee di uscita.

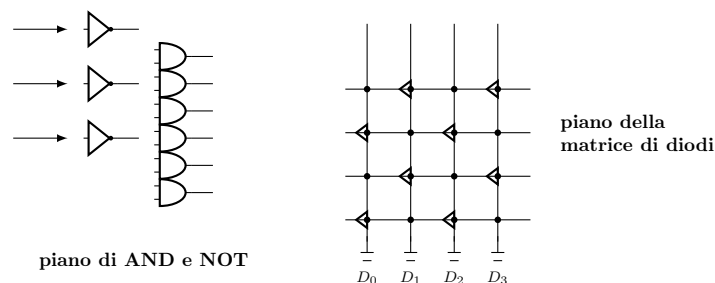


10.1.1 Come è fatta?

La ROM è costituita da un **piano di AND** e un **piano di OR**:



Per realizzare il circuito si usa la **matrice di diodi**.



La matrice di diodi usa **diodi** (non resistenze, così non c'è influenza fra linee diverse). Possiamo vedere $B_i = f(T_j)$; ad esempio:

$$B_0 = T_1 + T_3 + T_5 + T_7 + T_9 \quad (\text{righe collegate alla colonna } B_0)$$

Il circuito che implementa la ROM è: **decoder** + **encoder**.

10.1.2 Esempio: la tastiera

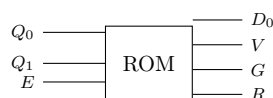
Ad ogni tasto si associa un codice BCD. La tastiera ha tasti $T_0 \dots T_9$ e uscite $B_3 B_2 B_1 B_0$ (codice BCD a 4 bit):

	T_9	T_8	T_7	T_6	T_5	T_4	T_3	T_2	T_1	T_0	B_3	B_2	B_1	B_0
	0	...	—	—					0	1	0	0	0	0
	...								...	...	...	...	...	...
	1	0	—						0	1	0	1	0	0

Gli incroci non sono collegati: i rami vengono uniti tramite la **matrice di diodi**.

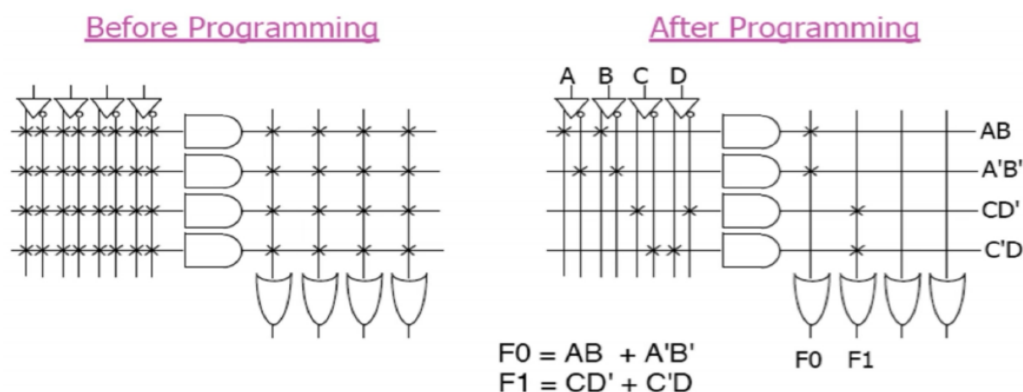


10.1.3 Esempio: la ROM del semaforo



10.1.4 Struttura della PLA: prima e dopo la programmazione

La PLA è composta da due matrici programmabili: un **piano AND** (che genera gli implicantti selezionati) e un **piano OR** (che combina gli implicantti per ciascuna uscita). Prima della programmazione *tutti* gli incroci sono connessi; la programmazione **brucia i diodi inutili**, lasciando solo le connessioni che implementano le funzioni desiderate.



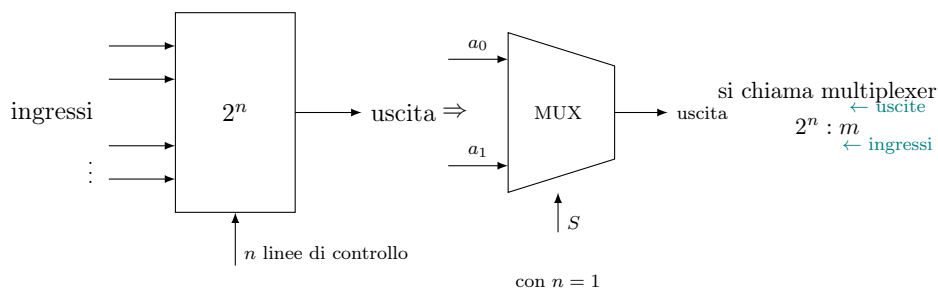
Esempio: con 4 ingressi A, B, C, D e 4 implicantti programmati (AB , $A'B'$, CD' , $C'D$) si implementano le due funzioni:

$$F_0 = AB + A'B' \quad F_1 = CD' + C'D = C \oplus D$$

La **PLA** risparmia area rispetto alla ROM perché genera *solo* gli implicantti necessari e non tutti i 2^n mintermini. Il numero di righe del piano AND è uguale al numero di implicantti distinti scelti (non a 2^n).

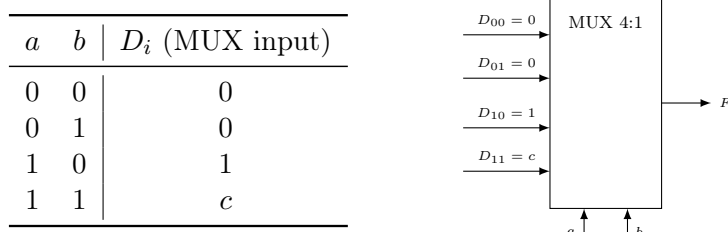
10.2 Multiplexer e demultiplexer

MUX (multiplexer) = circuito digitale che funziona similmente a un selettore, ma con 2^n ingressi, n linee di controllo e 1 uscita.

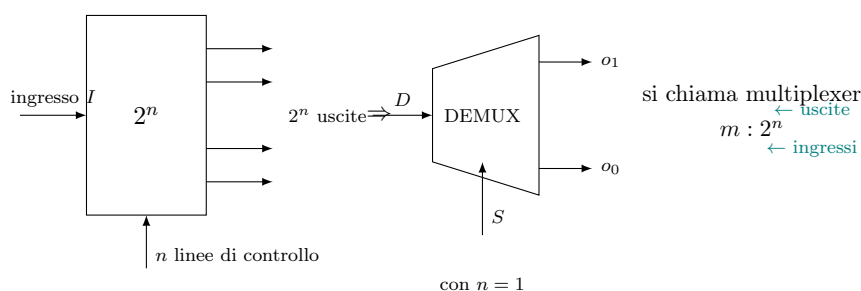


Ogni funzione a n variabili può essere implementata con un multiplexer $2^{n-1} : 1$.

Consideriamo $F(a, b, c) = a\bar{b} + abc$ e costruiamo la **tabella di verità funzionale** usando a e b come indirizzi del MUX 4:1 ($2^2 = 4$ ingressi) e c come eventuale ingresso dati:

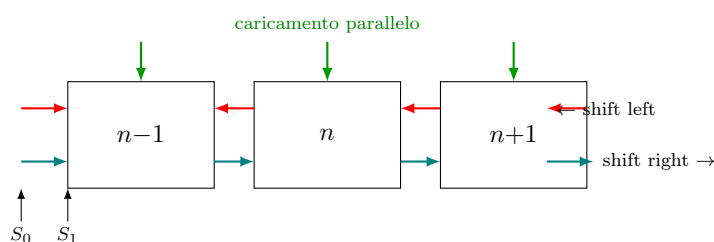


DEMUX (demultiplexer) = circuito digitale opposto al MUX, con 1 ingresso, n linee di controllo e 2^n uscite.



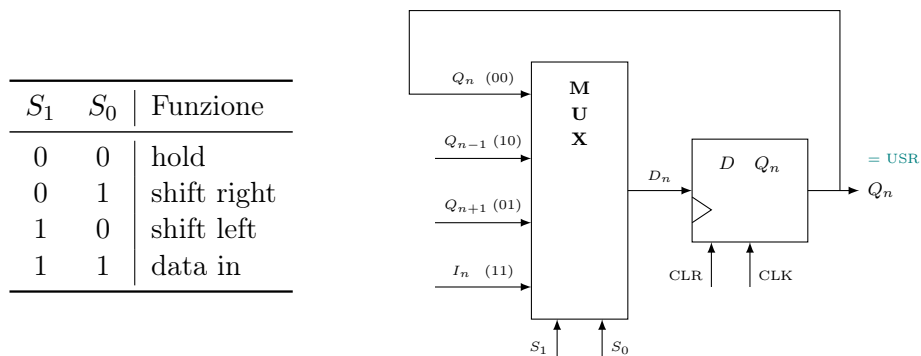
10.3 Universal Shift Register (USR)

USR (Universal Shift Register) = dispositivo che permette di mettere dati in parallelo o in serie e di spostarli a destra e sinistra.



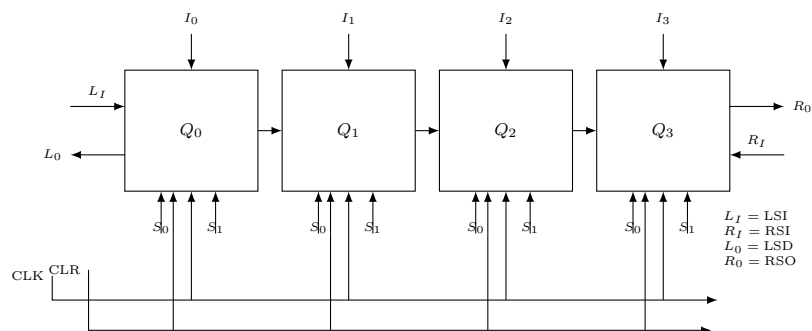
Vediamo come implementare ogni singola unità dell'USR.

Possiamo guidare le quattro funzioni con un **MUX 4:1** (2 indirizzi): HOLD, SHIFT LEFT, SHIFT RIGHT, DATA IN.



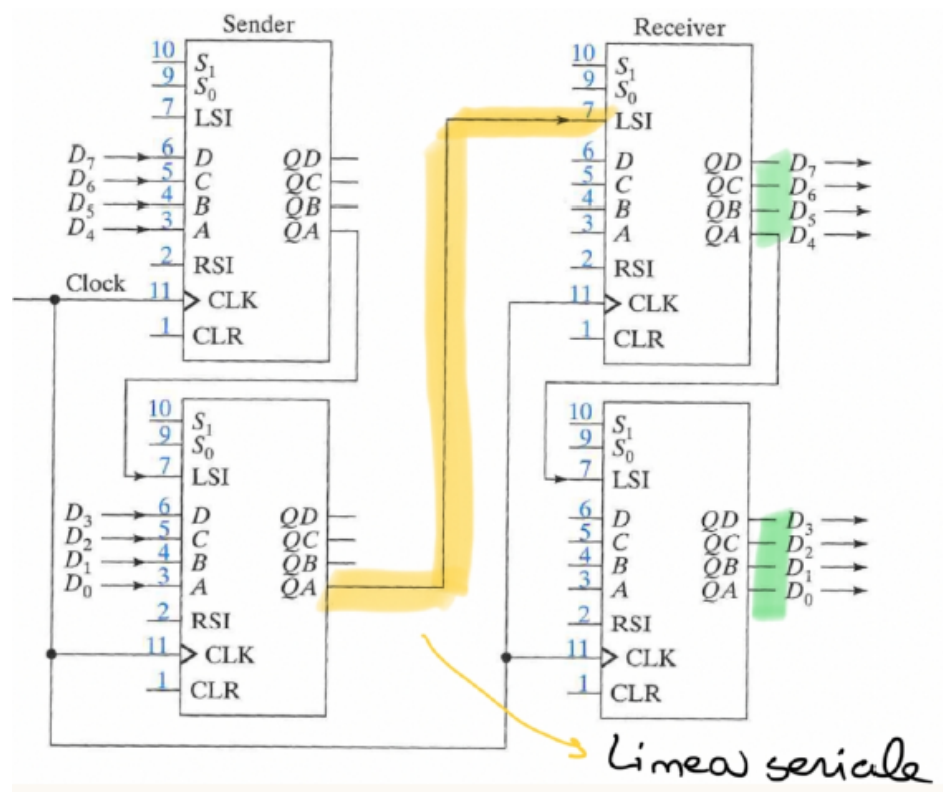
Se $CLR = 1$ si effettua un *RESET* a 0 sincrono.

10.3.1 Implementazione a 4 bit per trasmissione PISO



Il circuito permette 4 modalità operative selezionate da S_1S_0 : caricamento parallelo, shift left (seriale verso sinistra), shift right (seriale verso destra) e hold.

10.3.2 Applicazione: trasmissione seriale Sender–Receiver



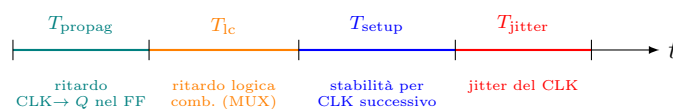
Il flusso dell'operazione è:

Caricamento nel Sender di $D_i \xrightarrow{\text{shift}}$ Trasmissione sulla linea seriale $\xrightarrow{\text{shift}}$ Lettura sulle uscite del Receiver

10.4 Vincoli temporali dell'USR e Jitter

La massima frequenza del clock per un USR corrisponde al minimo periodo del clock:

$$T_{\text{CLK,min}} = T_{\text{setup}} + T_{\text{propag}} + T_{\text{lc}} + T_{\text{jitter}}$$



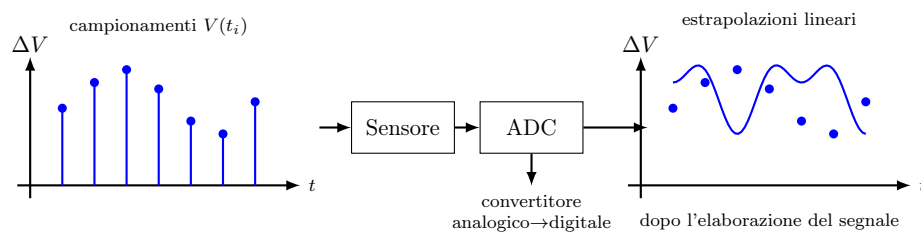
Jitter del CLK = Δ_{max} fra gli istanti di arrivo dei fronti di clock che dovrebbero giungere *contemporaneamente* a tutti i componenti del circuito, ma che in pratica non lo fanno a causa di variazioni fisiche (disadattamenti di linea, rumore, disomogeneità dei percorsi).

Capitolo 11. Conversione Digitale–Analogica e Analogica–Digitale

(15 maggio 2026)

11.1 Conversione digitale ↔ analogica

Il processo di acquisizione e ricostruzione del segnale si articola in due fasi:



Il quesito è: cosa succede fra il campionamento e la ricostruzione? La risposta è **sample and hold** → **convertitore analogico→digitale** → **convertitore digitale→analogico**, in sintesi:

Sensore $\xrightarrow{\text{ADC}}$ elaborazione digitale $\xrightarrow{\text{DAC}}$ uscita analogica

Definizioni di base:

- $n_{(\cdot)}$ = numero che dipende dai bit disponibili;
- ΔV = **granularità**, ovvero il passo minimo distinguibile;
- V_{ref} = valore massimo attingibile.

Funzione di trasferimento di un ADC: $n_{\text{out}} = V_{\text{in}}/\Delta V$

Funzione di trasferimento di un DAC: $V_{\text{out}} = n_{\text{in}} \cdot \Delta V$

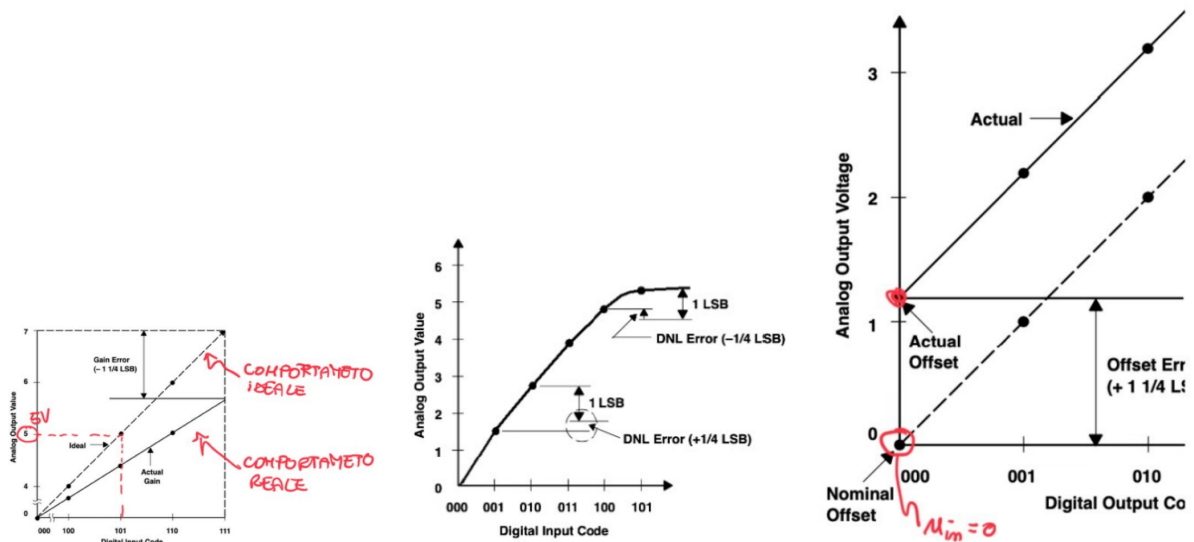
11.2 Caratteristiche del DAC

Precisione = determinata dal numero di bit e dall'intervallo di tensioni:

$$V_{\text{out}} = n_{\text{in}} \cdot \Delta V$$

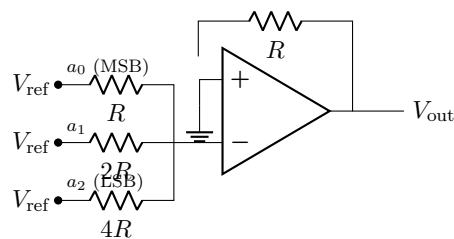
Ad esempio, con $n\text{-bit} = 3$ e $\Delta V = 1\text{ V}$: $n_{\text{in}} \in [0, 2^3-1] \Rightarrow V_{\text{out}} \in [0, 2^3-1]\text{ V}$.

La velocità di conversione è molto importante e dipende da diversi fattori: tipo di assestamento, architettura e numero di bit. Le tre principali sorgenti di errore nel DAC sono:



11.3 DAC a sommatore

L'architettura più semplice di DAC è il **DAC a sommatore pesato**: si usa un amplificatore operazionale invertente con resistenze pesate $R_i = R/2^i$ per ciascun bit. Le resistenze di ingresso sono collegate a V_{ref} (bit $a_i = 1$) o a massa (bit $a_i = 0$) tramite uno switch digitale per ciascun bit.



Nota: ogni ramo a_i è collegato a V_{ref} tramite uno switch digitale, che si chiude o si apre (collegando in alternativa a massa) a seconda del valore del bit; il simbolo del cursore $\updownarrow$ negli appunti originali indica proprio questo switch a monte dell'ingresso invertente.

La formula di uscita si ricava dalla sovrapposizione degli effetti:

$$V_{\text{out}} = -V_{\text{ref}} \left(\frac{R}{R_0} + \frac{R}{R_1} + \frac{R}{R_2} \right)$$

Se si pone $R_i = R/2^i$, allora $R/R_i = 2^i$ e quindi:

$$V_{\text{out}} = -V_{\text{ref}} \sum_i 2^i$$

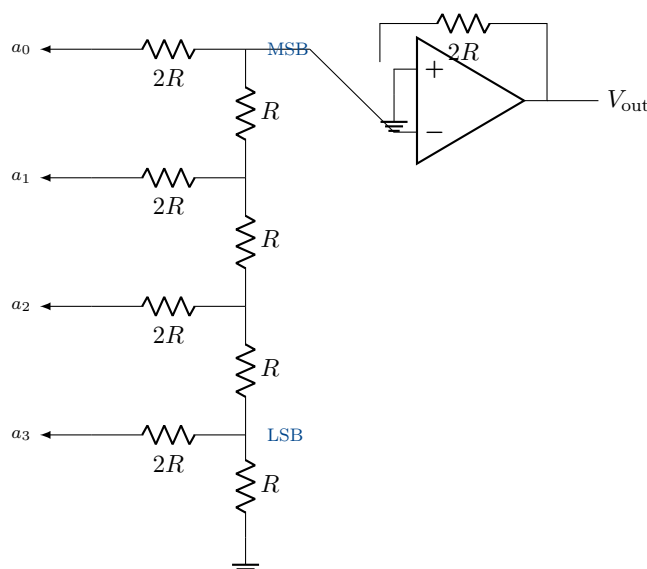
Considerando gli switch digitali, ogni bit a_i vale 0 (collegamento a terra) oppure 1 (collegamento a V_{ref}):

$$V_{\text{out}} = -V_{\text{ref}} \sum_i a_i 2^i$$

Limite pratico: supponendo 10 bit, $R_i = 100\text{k}\Omega/2^i$. Se la tolleranza è $\sigma_0 = 1\%$ e $R_0 = 1\text{k}\Omega$, i bit meno significativi vengono persi perché il loro valore di resistenza scende sotto la tolleranza stessa ($R_i < \sigma_0$). Questo DAC è quindi utilizzabile solo con pochi bit; altrimenti non è sufficientemente preciso.

11.4 DAC R-2R Ladder

Per avere un DAC più preciso si usa la tecnica **R-2R Ladder**, che genera pesi binari per i bit usando solo due valori di resistenza (R e $2R$):



Ogni bit a_i contribuisce all'uscita con un peso binario:

$$a_i \rightarrow V_{out,i} = -\frac{V_{ref}}{2^{i+1}} \Rightarrow \text{i pesi sono } \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots$$

$$V_{out} = -V_{ref} \sum_i \frac{a_i}{2^{i+1}}$$

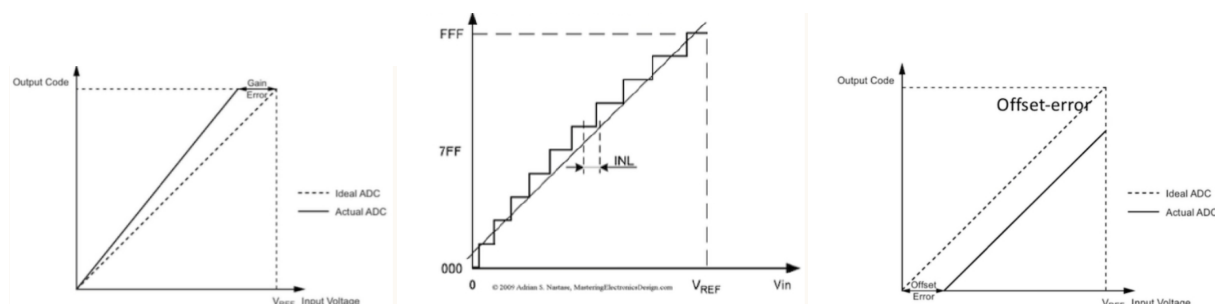
Esempio: con 4 bit, ingresso 1010_2 e $V_{ref} = 8\text{ V}$:

$$V_{out} = -8 \left(\frac{1}{2} + \frac{1}{8} \right) = -8 \cdot \frac{5}{8} = -5\text{ V}$$

11.5 Caratteristiche dell'ADC

- **Accuratezza** = insieme di tutte le sorgenti di errore, errore di quantizzazione compreso.
- **Errore di quantizzazione** = dipende dal tipo di ADC ed è dovuto alla discretizzazione del segnale durante la conversione; di solito vale 1 LSB oppure 0,5 LSB.
- **Tempo di conversione e funzionamento** sono altre caratteristiche importanti dell'ADC.

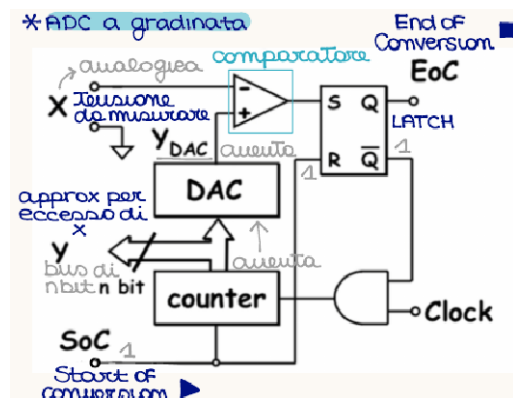
Analogamente al DAC, anche l'ADC presenta tre principali tipi di errore rispetto al comportamento ideale:



Capitolo 12. Convertitori Analogico-Digitali (ADC)

(15 maggio 2026 – cont.)

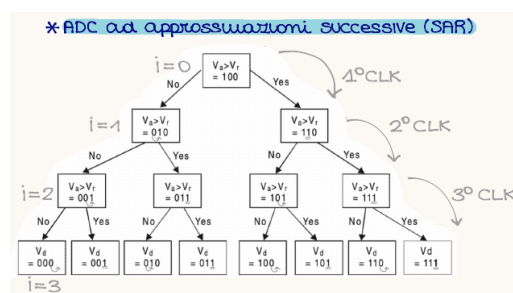
12.1 ADC a gradinata



- **Funzionamento del comparatore:** finché $X > Y_{DAC}$ il conteggio procede; appena $Y_{DAC} \geq X$ il comparatore cambia stato → il latch RS blocca il CLK e il counter.
- **Funzionamento del contatore:** genera $N_{in_2} \rightarrow N_{in_0}$ e aumenta la tensione.
- La granularità dipende da V_{REF} del DAC (cioè il ΔV con cui aumenta).
- Asintoticamente questo ADC deve fare 2^n confronti, dove n sono i livelli del DAC:

$$T_{conv} = 2^n T_{CLK}$$

12.2 ADC ad approssimazioni successive (SAR)

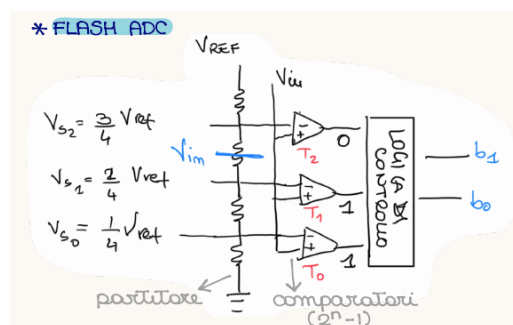


- Il valore di tensione da misurare si confronta con i successivi.
- Ogni conversione avviene in un ciclo di CLK. Se $n_{bit} = 3$ ci sono 3 cicli.
- **Yes:** metto un 1 nella posizione $i + 1$ se passo da i a $i + 1$.
- **No:** sposto l'1 dalla posizione i alla posizione $i + 1$ passando da i a $i + 1$.
- È una ricerca binaria: i confronti sono n , quindi:

$$T_{conv} = n T_{CLK}$$

- Funzionamento specifico alla pagina successiva.

12.3 Flash ADC



- La tensione viene confrontata contemporaneamente con n valori; poi un circuito combinatorio converte le uscite dei confronti in binario (1 = valori coperti da V_{in} ; 0 = valori più alti di V_{in}).
- Un confronto: istantaneo (fatto in parallelo) $\Rightarrow$ è il più veloce.
- La precisione è però molto minore di quella degli altri (servono tante resistenze, con i problemi già discussi).
- Risolvo coi mintermini, sapendo che se $j < i < k$ e $T_i = 1$ allora $T_j = 1$ e $T_k = 0$:

$$b_0 = T_2 + T_0 \cdot T_1$$

$$b_1 = T_2 + T_1 \cdot \overline{T_2}$$

12.4 Osservazioni generali

- L'errore di quantizzazione è pari a $0,5 V_{REF}/2^n$.

12.5 ADC dell'AD2

- Ci sono 14 bit $\Rightarrow 2^{14}$ possibili valori.
- Se scelgo la scala 0.5 V/div l'intervallo in cui misuro è $(0, 0,5 \cdot 10) \text{ V} = (0, 5) \text{ V}$, e la quantizzazione è:

$$\frac{5 \text{ V}}{2^{14}} \approx 0.3 \text{ mV}$$

12.5.1 ADC ad approssimazioni successive (SAR): analisi del circuito a 3 bit

Il convertitore analogico-digitale ad *approssimazioni successive* (in inglese **S**uccessive **A**pproximation **R**egister, SAR) determina il codice digitale corrispondente ad una tensione analogica V_a eseguendo un confronto "dicotomico" bit a bit, a partire dal bit più significativo (MSB) fino al meno significativo (LSB). Il circuito visto a lezione realizza un SAR a 3 bit ed è composto dai seguenti blocchi funzionali:

- **Ring Counter** (registro a scorrimento circolare) costituito da 5 flip-flop $S_0, \dots, S_4$: genera, un colpo di clock alla volta, un impulso che si sposta da uno stadio al successivo. Questo impulso scandisce il tempo di test di ciascun bit (MSB $\rightarrow$ LSB) e determina quando la conversione è terminata.
- **Registro dei bit di prova** composto da 3 flip-flop Q_2, Q_1, Q_0 (MSB $\rightarrow$ LSB): contiene, istante per istante, il codice binario di tentativo.
- **DAC** (convertitore digitale-analogico): converte il codice presente in $Q_2Q_1Q_0$ nella tensione di confronto V_{DAC} .
- **Comparatore analogico**: confronta V_a (tensione incognita da convertire) con V_{DAC} .

- **Logica combinatoria (porte AND/OR):** in base all'uscita del comparatore e allo stadio attivo del Ring Counter, decide se mantenere a 1 oppure resettare a 0 il bit appena testato.

È importante notare che il Ring Counter e il registro dei bit di prova lavorano *in parallelo*: mentre il Ring Counter “punta” al bit che si sta testando in quel colpo di clock, il registro $Q_2Q_1Q_0$ contiene il codice di prova corrente, che viene aggiornato bit dopo bit.

Fase 1 – Inizializzazione (prima del primo fronte di clock)

Prima dell'arrivo del primo fronte di clock il circuito viene inizializzato nel seguente modo:

1. Si invia il segnale di **CLEAR** (attivo basso) contemporaneamente a tutti i flip-flop del registro dati Q_2, Q_1, Q_0 : tutte le uscite vengono forzate a 0.
2. Contemporaneamente si setta $S_0 = 1$ (primo stadio del Ring Counter) tramite ingresso asincrono, mentre tutti gli altri stadi $S_1, \dots, S_4$ vengono inizializzati a 0.
3. Poiché il registro dati è tutto a 0, l'ingresso del DAC è 000, quindi l'uscita del DAC è

$$V_{DAC} = 0 \text{ V.}$$

Lo stato del sistema subito prima del primo fronte di clock è dunque:

$$(S_4S_3S_2S_1S_0) = (00001), \quad (Q_2Q_1Q_0) = (000), \quad V_{DAC} = 0 \text{ V.}$$

Fase 2 – Subito dopo il primo fronte di clock

Al primo fronte di clock accadono due cose contemporaneamente:

- Il Ring Counter avanza di uno stadio: $S_0 \rightarrow 0, S_1 \rightarrow 1$. Questo impulso su S_1 abilita la scrittura del bit più significativo Q_2 .
- La logica combinatoria pone *a priori* $Q_2 = 1$ (si “prova” il bit più significativo settandolo a 1): il nuovo codice di prova è $Q_2Q_1Q_0 = 100$.

Con il nuovo codice in ingresso, il DAC produce la tensione corrispondente a $n = 4$ (in decimale):

$$V_{DAC} = V(n=4) \quad (\text{tensione corrispondente al codice } 100).$$

A questo punto il comparatore confronta V_a con V_{DAC} : il risultato di questo confronto sarà disponibile e utilizzato al fronte di clock *successivo* per decidere se mantenere $Q_2 = 1$ oppure resettarlo a 0.

Esempio numerico

Supponiamo che il DAC abbia una risoluzione $\Delta V = 1 \text{ V}$ sull'intervallo $[0, 7] \text{ V}$ (3 bit, 8 livelli) e che la tensione incognita sia

$$V_a = 5.5 \text{ V.}$$

Dopo il primo fronte di clock il registro contiene $Q_2Q_1Q_0 = 100$, quindi

$$V_{DAC} = 4 \text{ V.}$$

Il comparatore verifica se $V_a > V_{DAC}$ (equivalentemente, se V_a supera la metà del fondo scala, $V_{max}/2$):

$$V_a = 5.5 \text{ V} > V_{DAC} = 4 \text{ V} \implies \text{SI.}$$

Poiché il confronto ha dato esito positivo, il bit Q_2 appena testato viene **mantenuto a 1** (l'approssimazione “per difetto” era corretta: la tensione reale è maggiore di quella generata dal DAC con quel bit a 1, quindi quel bit resta valido nel codice finale).

Fase 4 – Subito dopo il secondo fronte di clock

Al secondo fronte di clock:

- Il Ring Counter avanza ancora: $S_1 \rightarrow 0$, $S_2 \rightarrow 1$. L'impulso su S_2 abilita ora la scrittura del bit successivo Q_1 .
- Il risultato del confronto precedente (esito “SI”, $V_a > V_{DAC}$) viene usato dalla logica combinatoria per **confermare** $Q_2 = 1$ definitivamente.
- Contemporaneamente si prova il nuovo bit ponendo $Q_1 = 1$: il codice di prova diventa $Q_2Q_1Q_0 = 110$.

Il DAC aggiorna quindi la propria uscita al nuovo codice:

$$V_{DAC} = 4 \text{ V} + 2 \text{ V} = 6 \text{ V}.$$

Si esegue un nuovo confronto:

$$V_a = 5.5 \text{ V} < V_{DAC} = 6 \text{ V} \implies \text{esito negativo.}$$

Questa volta il bit appena provato (Q_1) risulterà, al fronte di clock successivo, **resettato a 0** (il DAC ha superato V_a , quindi quel bit non può essere 1 nel codice finale): il codice torna a 100 per quanto riguarda Q_2Q_1 , e si passerà a testare l'ultimo bit Q_0 .

Algoritmo generale (n bit)

Il procedimento si generalizza a un ADC-SAR a n bit come segue: per ogni bit, dal MSB al LSB,

1. il Ring Counter abilita la scrittura del bit corrente;
2. la logica pone a 1 (in via di prova) il bit corrente, lasciando invariati i bit già decisi in precedenza;
3. il DAC converte il nuovo codice di prova in V_{DAC} ;
4. il comparatore confronta V_a con V_{DAC} ;
5. al fronte di clock successivo, in base al risultato del confronto:
 - se $V_a > V_{DAC}$: il bit resta a 1 (viene confermato);
 - se $V_a < V_{DAC}$: il bit viene riportato a 0.

Il processo richiede quindi n colpi di clock per determinare i bit, più un colpo aggiuntivo di inizializzazione/lettura: per questo il Ring Counter dell'esempio ha 5 stadi ($S_0, \dots, S_4$) per un ADC a 3 bit (1 stadio di start + 3 stadi di conversione + 1 stadio di fine conversione/lettura del risultato).

Tabella riassuntiva dell'esempio ($V_a = 5.5 \text{ V}$)

Fronte di clock	Bit testato	Codice di prova $Q_2Q_1Q_0$	V_{DAC}	Confronto $V_a \geq V_{DAC}$
1 (init.)	Q_2	100	4 V	$5.5 > 4 \Rightarrow$ mantieni
2	Q_1	110	6 V	$5.5 < 6 \Rightarrow$ resetta
3	Q_0	101	5 V	$5.5 > 5 \Rightarrow$ mantieni

Il codice digitale finale è dunque $Q_2Q_1Q_0 = 101$ (decimale 5), corrispondente a $V_{DAC} = 5 \text{ V}$: il valore più vicino per difetto a $V_a = 5.5 \text{ V}$ ottenibile con una risoluzione di 1 V (errore di quantizzazione = 0.5 V , cioè metà del passo, come atteso).

Fase 5 – Valutazione del confronto (subito dopo il secondo fronte)

Nello stesso ciclo di clock in cui Q_1 è stato posto a 1 (Fase 4), il comparatore valuta il nuovo confronto tra V_a e la V_{DAC} appena aggiornata:

$$V_a = 5.5 \text{ V} \quad \text{vs} \quad V_{DAC} = 6 \text{ V}.$$

Poiché

$$V_a = 5.5 \text{ V} < V_{DAC} = 6 \text{ V},$$

l'esito del confronto è **negativo**: il bit Q_1 appena provato *non* sarà confermato. Questo risultato viene memorizzato dalla logica combinatoria e sarà applicato al registro dati al fronte di clock *successivo* (Fase 6): la fase di “confronto” è quindi concettualmente distinta dalla fase di “scrittura”, anche se entrambe avvengono nello stesso periodo di clock.

Fase 6 – Subito dopo il terzo fronte di clock

Al terzo fronte di clock accadono, come nei casi precedenti, due cose:

- Il Ring Counter avanza: $S_2 \rightarrow 0$, $S_3 \rightarrow 1$. L'impulso su S_3 abilita ora la scrittura del bit meno significativo Q_0 .
- In base al risultato negativo della Fase 5, il bit Q_1 viene **resettato a 0**: il codice torna ad avere $Q_2Q_1 = 10$.
- Contemporaneamente si prova il nuovo bit ponendo $Q_0 = 1$: il codice di prova diventa $Q_2Q_1Q_0 = 101$.

Il DAC aggiorna quindi la propria uscita:

$$V_{DAC} = 4 \text{ V} + 0 \text{ V} + 1 \text{ V} = 5 \text{ V}.$$

Fase 7 – Confronto sull'ultimo bit (LSB)

Si esegue l'ultimo confronto della conversione:

$$V_a = 5.5 \text{ V} > V_{DAC} = 5 \text{ V} \quad \implies \quad \text{esito } \mathbf{positivo} \text{ (SI)}.$$

Il bit Q_0 appena provato viene quindi **mantenuto a 1**. A questo punto tutti e tre i bit sono stati testati e il codice binario è stato completamente determinato:

$$Q_2Q_1Q_0 = 101 \quad (= 5 \text{ in decimale}), \quad V_{DAC} = 5 \text{ V}.$$

Fase 8 – Subito dopo il quarto fronte di clock: fine conversione

Al quarto fronte di clock il Ring Counter avanza all'ultimo stadio ($S_3 \rightarrow 0$, $S_4 \rightarrow 1$). Questo impulso non serve più a testare un bit (tutti e 3 i bit sono già stati decisi), ma segnala la **fine della conversione**:

- Vengono attivate le porte di uscita G_A, G_B, G_C , che trasferiscono il contenuto del registro $Q_2Q_1Q_0 = 101$ alle uscite digitali finali del convertitore D_A, D_B, D_C (mostrate nello schema come $-1, -0, -1$, cioè $1, 0, 1$).
- Il risultato $101_2 = 5$ rappresenta un **arrotondamento per difetto** di $V_a = 5.5$ V rispetto alla risoluzione del DAC ($\Delta V = 1$ V): l'errore di quantizzazione vale $5.5 - 5 = 0.5$ V, pari a metà passo, come atteso per un ADC con questa risoluzione.

Il valore digitale $N_{ADC} = 101$ resta stabile in uscita (latched) finché non verrà completata una nuova conversione: “ N_{ADC} rimane costante finché il prossimo valore verrà trasferito”.

Fase 9 – Avvio di una nuova conversione

Quando si presenta un nuovo valore analogico da convertire, ad esempio

$$V_a = 3.5 \text{ V},$$

al quinto fronte di clock (che coincide, per il Ring Counter, con un nuovo primo fronte del ciclo successivo) accade quanto segue:

- Viene inviato nuovamente il segnale di **CLEAR** ai flip-flop del registro dati Q_0, Q_1 (e implicitamente Q_2): il registro si azzerà per iniziare una nuova ricerca dicotomica.
- Il Ring Counter viene resettato: $S_4 \rightarrow 0$, $S_0 \rightarrow 1$ di nuovo, esattamente come nella Fase 1 iniziale.
- Si ricomincia quindi il ciclo di conversione da capo, testando nuovamente il MSB Q_2 : il registro di prova riparte da $Q_2Q_1Q_0 = 100$ e $V_{DAC} = 4$ V.

Le uscite finali $D_AD_BD_C$ (risultato della *conversione precedente*, 101) restano invariate in questa fase, poiché vengono aggiornate solo al termine della *nuova* conversione (di nuovo dopo 4 fronti di clock): questo è il motivo per cui il registro di uscita e il registro interno di lavoro sono tenuti separati nello schema.

Tabella riassuntiva completa (aggiornata)

Fronte	Ring Counter attivo	Bit testato	Codice $Q_2Q_1Q_0$	V_{DAC}	Esito confronto
–	S_0	(init.)	000	0 V	–
1	S_1	Q_2	100	4 V	$5.5 > 4$: mantieni
2	S_2	Q_1	110	6 V	$5.5 < 6$: resetta
3	S_3	Q_0	101	5 V	$5.5 > 5$: mantieni
4	S_4	–	101	5 V	fine conversione, latch uscite

Risultato finale: $D_AD_BD_C = 101_2 = 5$, con $V_{DAC} = 5$ V $\approx V_a = 5.5$ V (arrotondamento per difetto).